



Licenciatura Ciências de Computação
Mestrado Integrado Eng^a. Informática
Mestrado Integrado Eng^a. Física

2020/21

A.J.Proença

Tema

ISA do IA-32

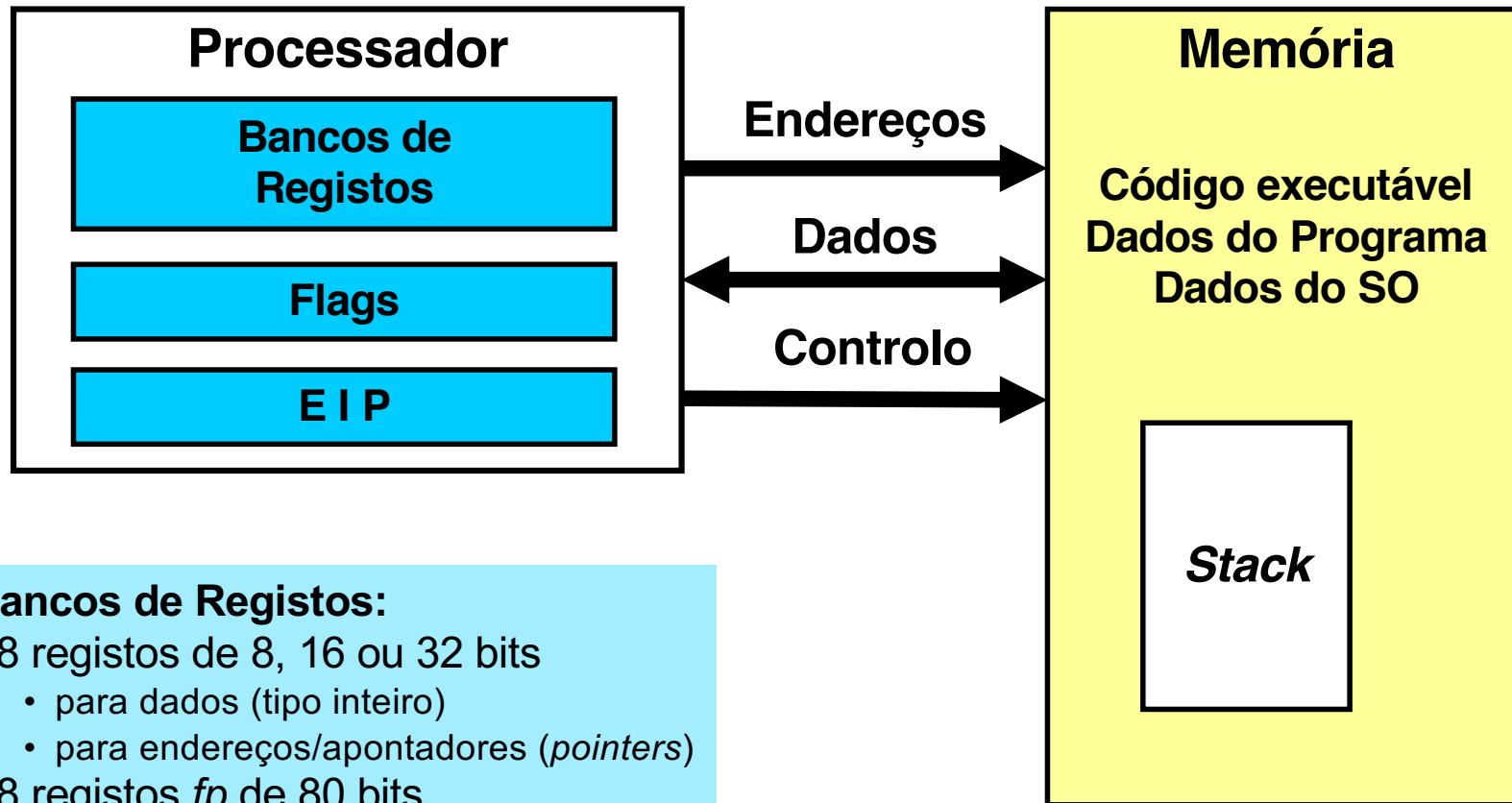
Análise do Instruction Set Architecture (1)



Estrutura do tema ISA do IA-32

1. Desenvolvimento de programas no IA-32 em Linux
2. Acesso a operandos e operações
3. Suporte a estruturas de controlo
4. Suporte à invocação/regresso de funções
5. Análise comparativa: IA-32 vs. x86-64 e RISC (MIPS e ARM)
6. Acesso e manipulação de dados estruturados

O modelo Processador-Memória no IA-32 (visão do programador)



Bancos de Registos:

- 8 registos de 8, 16 ou 32 bits
 - para dados (tipo inteiro)
 - para endereços/apontadores (*pointers*)
- 8 registos *fp* de 80 bits

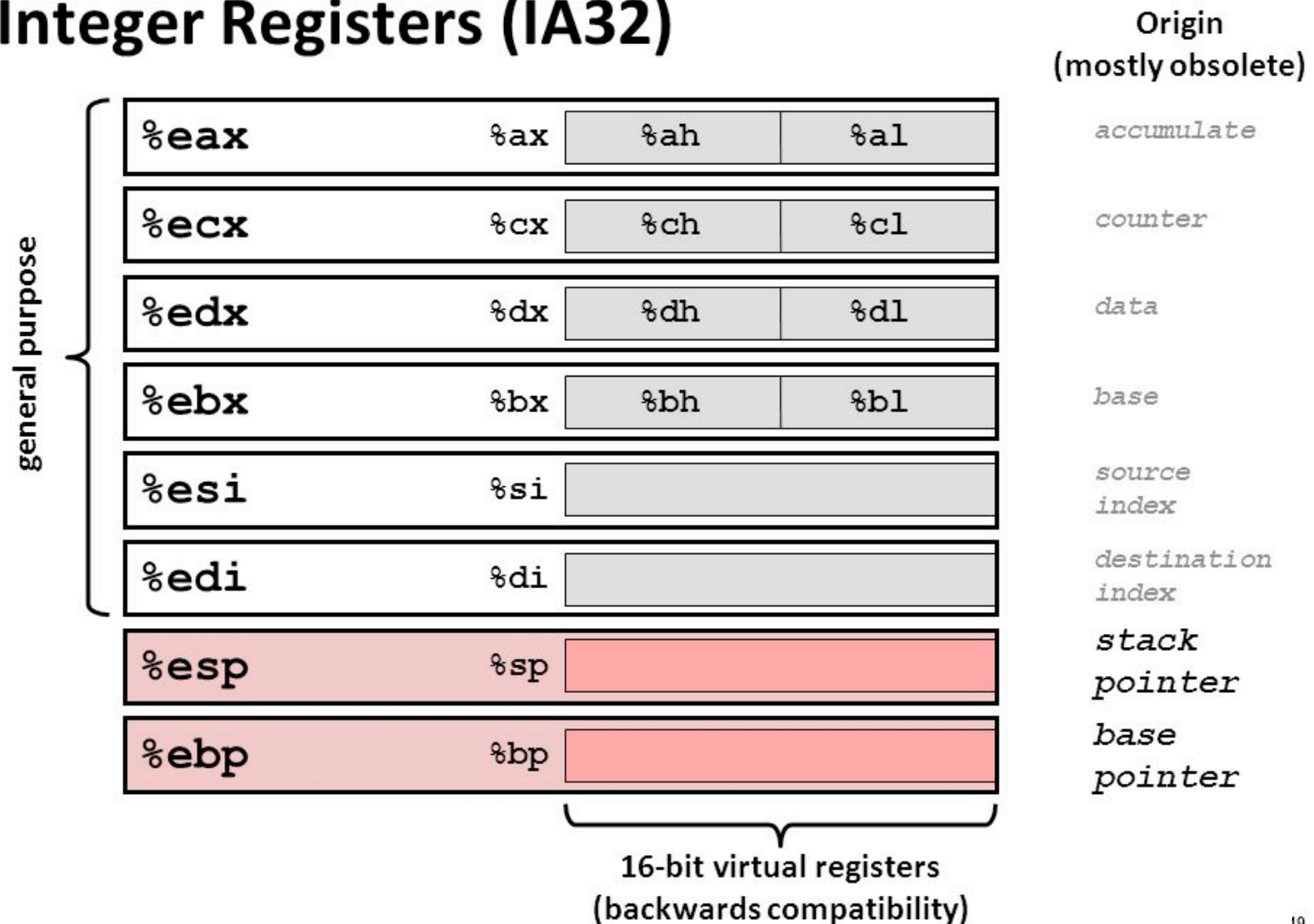
Flags:

- estado da última op aritm/lóg

O banco de registos para inteiros / apontadores



Integer Registers (IA32)



Representação de operandos no IA-32



- Tamanhos de objetos em C (em *bytes*)

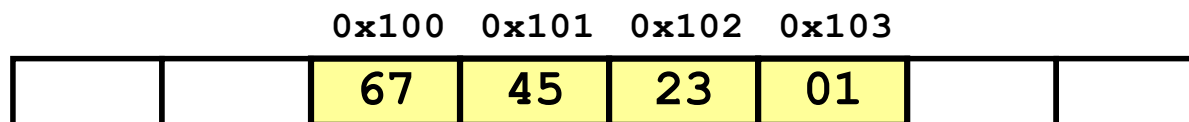
Declaração em C	Designação Intel	Tamanho IA-32
char	byte	1
short	word	2
int	double word	4
long int	double word	4
float	single precision	4
double	double precision	8
long double	extended precision	10/12
char * (ou qq outro apontador)	double word	4

- Ordenação dos *bytes* na memória

- O IA-32 é um processador *little endian*

- Exemplo:

- valor de **var** (0x01234567) na memória, cujo endereço &var é 0x100



Tipos de instruções básicas no IA-32



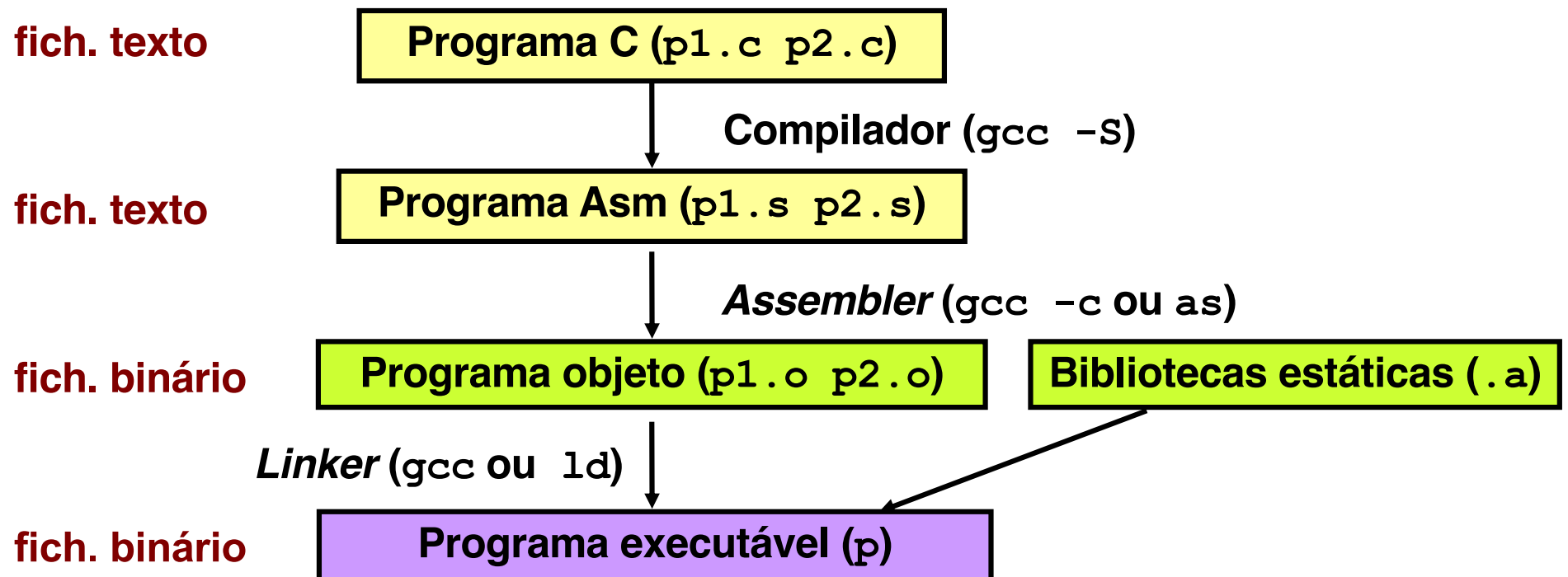
Operações primitivas:

- Efetuar operações aritméticas/lógicas
com dados em registo ou em memória
 - dados do tipo *integer* de 1, 2 ou 4 *bytes*; em complemento p/ 2
 - dados em formato *fp* de 4, 8 ou 10 *bytes*; precisão simples ou dupla
 - operações só com dados escalares; op's com vetores possível
 - *arrays* ou *structures*; *bytes* continuamente alocados em memória
- Transferir dados entre células de memória e um registo
 - carregar (*load*) em registo dados copiados da memória
 - armazenar (*store*) na memória valores guardados em registo
- Transferir o controlo da execução das instruções
 - saltos incondicionais para outras partes do programa/módulo
 - saltos ramificados (*branches*) condicionais
 - saltos incondicionais para/de funções/procedimentos

Conversão de um programa em C em código executável (exemplo)



- Código C nos ficheiros : `p1.c` `p2.c`
- Comando para a "compilação": `gcc -O2 p1.c p2.c -o p`
 - usa otimizações (`-O2`)
 - coloca binário resultante no ficheiro `p` (`-o p`)



A compilação de C para assembly (exemplo)



Código C

```
int sum(int x, int y)
{
    int t = x+y;
    return t;
}
```

Assembly gerado

```
_sum:
    pushl    %ebp
    movl     %esp, %ebp
    movl     12(%ebp), %eax
    addl     8(%ebp), %eax
    movl     %ebp, %esp
    popl     %ebp
    ret
```

gcc -O2 -S p2.c

p2.s

De assembly para objeto e executável (exemplo)



Assembly

```
_sum:
    pushl    %ebp
    movl     %esp, %ebp
    movl     12(%ebp), %eax
    addl     8(%ebp), %eax
    movl     %ebp, %esp
    popl     %ebp
    ret
```

p2.s

p2.o

Código binário

```
0x401040 <sum>:
    0x55
    0x89 • Começa no
    0xe5   endereço
    0x8b   0x401040
    0x45
    0x0c • Total 13
    0x03   bytes
    0x45
    0x08
    0x89 • Cada
    0xec   instrução
    0x5d   1, 2, ou 3
    0xc3   bytes
```

Papel do linker

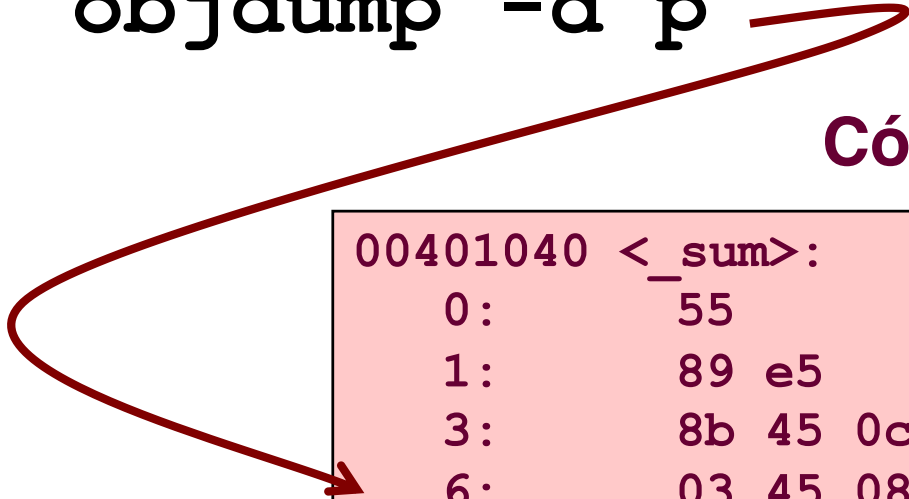
- Resolve as referências entre ficheiros
- Junta as *static run-time libraries*
 - E.g., código para `malloc`, `printf`
- Algumas bibliotecas são *dynamically linked*
 - E.g., junção ocorre no início da execução

Desmontagem de código binário executável (exemplo)



objdump -d p

Código binário desmontado



```
00401040 <_sum>:
  0:      55                push    %ebp
  1:      89 e5            mov     %esp,%ebp
  3:      8b 45 0c         mov     0xc(%ebp),%eax
  6:      03 45 08         add     0x8(%ebp),%eax
  9:      89 ec            mov     %ebp,%esp
  b:      5d              pop     %ebp
  c:      c3              ret
  d:      8d 76 00        lea     0x0(%esi),%esi
```

Método alternativo de análise do código binário executável (exemplo)



Entrar primeiro no depurador `gdb` : `gdb p` `e...`

- examinar apenas alguns *bytes* : `x/13xb sum`

```
0x401040<sum>: 0x55 0x89 0xe5 0x8b 0x45 0x0c 0x03 0x45
0x401048<sum+8>: 0x08 0x89 0xec 0x5d 0xc3
```

... OU

- proceder à desmontagem do código : `disassemble sum`

```
0x401040 <sum>:      push    %ebp
0x401041 <sum+1>:      mov     %esp, %ebp
0x401043 <sum+3>:      mov     0xc(%ebp), %eax
0x401046 <sum+6>:      add     0x8(%ebp), %eax
0x401049 <sum+9>:      mov     %ebp, %esp
0x40104b <sum+11>:     pop     %ebp
0x40104c <sum+12>:     ret
0x40104d <sum+13>:     lea     0x0(%esi), %esi
```

Que código pode ser desmontado?



Qualquer ficheiro que possa ser interpretado como código executável
– o *disassembler* examina os *bytes* e reconstrói o código em *assembly*

```
% objdump -d WINWORD.EXE

WINWORD.EXE:      file format pei-i386

No symbols in "WINWORD.EXE".
Disassembly of section .text:

30001000 <.text>:
30001000:  55                push    %ebp
30001001:  8b ec            mov     %esp, %ebp
30001003:  6a ff            push    $0xffffffff
30001005:  68 90 10 00 30    push    $0x30001090
3000100a:  68 91 dc 4c 30    push    $0x304cdc91
```

Análise do Instruction Set Architecture (2)



Estrutura do tema ISA do IA-32

1. Desenvolvimento de programas no IA-32 em Linux
2. Acesso a operandos e operações
3. Suporte a estruturas de controlo
4. Suporte à invocação/regresso de funções
5. Análise comparativa: IA-32 vs. x86-64 e RISC (MIPS e ARM)
6. Acesso e manipulação de dados estruturados

Acesso a operandos no IA-32: sua localização e modos de acesso



Localização de operandos no IA-32

- valores de constantes (ou valores imediatos)
 - incluídos na instrução, i.e., no Reg. Instrução (IR)
- variáveis escalares
 - sempre que possível, em registos (inteiros/apontadores) / *fp* ; se não...
 - na memória (inclui *stack*)
- variáveis estruturadas
 - sempre na memória, em células contíguas

Modos de acesso a operandos no IA-32

- em instruções de transferência de informação
 - instrução mais comum: `movx` , sendo *x* o tamanho (*b*, *w*, *l*)
 - algumas instruções atualizam apontadores (por ex.: `push`, `pop`)
- em operações aritméticas/lógicas

Análise de uma instrução de transferência de informação



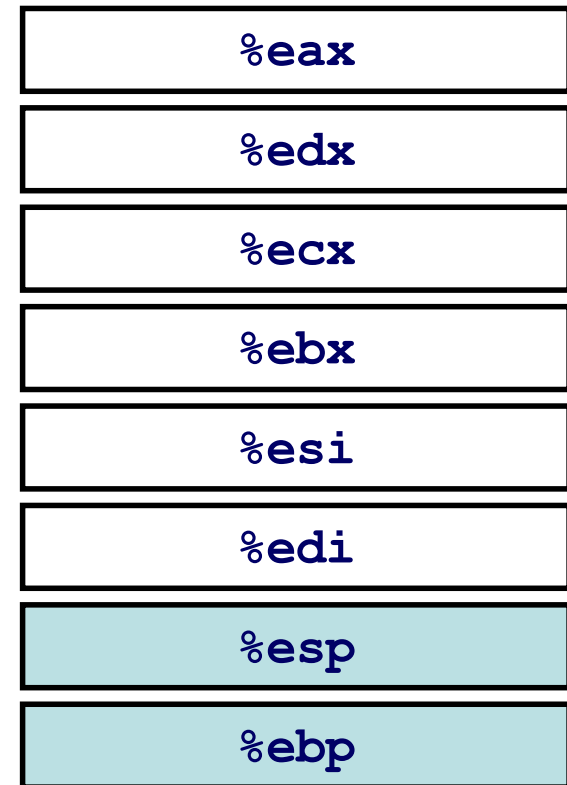
• Transferência simples

`movl Source, Dest`

- move um valor de 4 bytes (“long”)
- instrução mais comum em código IA-32

• Tipos de operandos

- imediato: valor constante do tipo inteiro
 - como a constante em C, mas com prefixo ‘\$’
 - ex.: \$0x400, \$-533
 - codificado com 4 bytes (em `movl`)
- em registo: um de 8 registos inteiros
 - mas... %esp e %ebp estão reservados...
 - e outros poderão ser usados implicitamente...
- em memória: 4 bytes consecutivos de memória (em `movl`)
 - vários modos de especificar a sua localização (o endereço)...



Análise da localização dos operandos na instrução *movl*



	Fonte	Destino		Equivalente em C
movl	Imm	Reg	<code>movl \$0x4,%eax</code>	<code>temp = 0x4;</code>
		Mem	<code>movl \$-147, (%eax)</code>	<code>*p = -147;</code>
	Reg	Reg	<code>movl %eax,%edx</code>	<code>temp2 = temp1;</code>
		Mem	<code>movl %eax, (%edx)</code>	<code>*p = temp;</code>
	Mem	Reg	<code>movl (%eax), %edx</code>	<code>temp = *p;</code>
		Mem	não é possível no IA-32 efetuar transferências memória-memória com uma só instrução	

Modos de endereçamento à memória no IA-32 (1)



- **Indireto (*normal*)** **(R)** $\text{Mem}[\text{Reg}[\text{R}]]$
 - conteúdo do registo **R** especifica o endereço de memória

`movl (%ecx), %eax`

- **Deslocamento** **D(R)** $\text{Mem}[\text{Reg}[\text{R}] + \text{D}]$
 - conteúdo do registo **R** especifica início da região de memória
 - deslocamento c^{te} **D** especifica distância do início (em *bytes*)

`movl -8(%ebp), %edx`

Exemplo de utilização de modos simples de endereçamento à memória no IA-32 (1)



```
void swap(int *xp, int *yp)
{
    int t0 = *xp;
    int t1 = *yp;
    *xp = t1;
    *yp = t0;
}
```

swap:

```
    pushl %ebp
    movl  %esp,%ebp
    pushl %ebx
```

Arranque

```
    movl 12(%ebp),%ecx
    movl 8(%ebp),%edx
    movl (%ecx),%eax
    movl (%edx),%ebx
    movl %eax, (%edx)
    movl %ebx, (%ecx)
```

Corpo

```
    movl -4(%ebp),%ebx
    movl %ebp,%esp
    popl %ebp
    ret
```

Término

Exemplo de utilização de modos simples de endereçamento à memória no IA-32 (2)



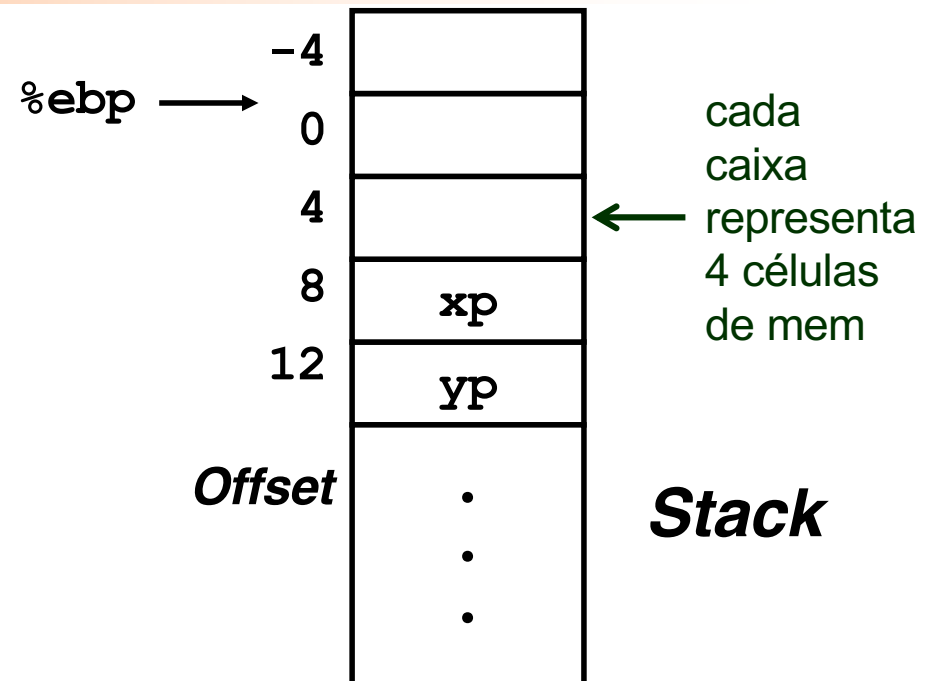
```
void swap(int *xp, int *yp)
{
    int t0 = *xp;
    int t1 = *yp;
    *xp = t1;
    *yp = t0;
}
```

Registo Variável

%ecx	yp
%edx	xp
%eax	t1
%ebx	t0

Corpo

<code>movl 12(%ebp), %ecx</code>	<code># ecx = yp</code>
<code>movl 8(%ebp), %edx</code>	<code># edx = xp</code>
<code>movl (%ecx), %eax</code>	<code># eax = *yp (t1)</code>
<code>movl (%edx), %ebx</code>	<code># ebx = *xp (t0)</code>
<code>movl %eax, (%edx)</code>	<code># *xp = eax</code>
<code>movl %ebx, (%ecx)</code>	<code># *yp = ebx</code>



Exemplo de utilização de modos simples de endereçamento à memória no IA-32 (3)



Corpo

```

movl 12(%ebp), %ecx # ecx = yp
movl 8(%ebp), %edx  # edx = xp
movl (%ecx), %eax   # eax = *yp (t1)
movl (%edx), %ebx   # ebx = *xp (t0)
movl %eax, (%edx)   # *xp = eax
movl %ebx, (%ecx)   # *yp = ebx
    
```

%eax	
%edx	
%ecx	
%ebx	
%esi	
%edi	
%esp	
%ebp	0x104

	Offset		Endereço
	-4		0x100
%ebp →			0x104
	4		0x108
xp	8	0x124	0x10c
yp	12	0x120	0x110
			0x114
			0x118
			0x11c
		456	0x120
		123	0x124

← 4 →
células

Exemplo de utilização de modos simples de endereçamento à memória no IA-32 (4)



Corpo

```

movl 12(%ebp), %ecx # ecx = yp
movl 8(%ebp), %edx  # edx = xp
movl (%ecx), %eax   # eax = *yp (t1)
movl (%edx), %ebx   # ebx = *xp (t0)
movl %eax, (%edx)   # *xp = eax
movl %ebx, (%ecx)   # *yp = ebx
    
```

%eax	
%edx	
%ecx	0x120
%ebx	
%esi	
%edi	
%esp	
%ebp	0x104

	Offset		Endereço
	-4		0x100
%ebp →			0x104
	4		0x108
xp	8	0x124	0x10c
yp	12	0x120	0x110
			0x114
			0x118
			0x11c
		456	0x120
		123	0x124

← 4 →
células

Exemplo de utilização de modos simples de endereçamento à memória no IA-32 (5)



Corpo

```

movl 12(%ebp), %ecx # ecx = yp
movl 8(%ebp), %edx  # edx = xp
movl (%ecx), %eax   # eax = *yp (t1)
movl (%edx), %ebx   # ebx = *xp (t0)
movl %eax, (%edx)   # *xp = eax
movl %ebx, (%ecx)   # *yp = ebx
    
```

%eax	
%edx	0x124
%ecx	0x120
%ebx	
%esi	
%edi	
%esp	
%ebp	0x104

	Offset		Endereço
	-4		0x100
%ebp →			0x104
	4		0x108
xp	8	0x124	0x10c
yp	12	0x120	0x110
			0x114
			0x118
			0x11c
		456	0x120
		123	0x124

← 4 →
células

Exemplo de utilização de modos simples de endereçamento à memória no IA-32 (6)



Corpo

```

movl 12(%ebp), %ecx # ecx = yp
movl 8(%ebp), %edx  # edx = xp
movl (%ecx), %eax   # eax = *yp (t1)
movl (%edx), %ebx   # ebx = *xp (t0)
movl %eax, (%edx)   # *xp = eax
movl %ebx, (%ecx)   # *yp = ebx
    
```

%eax	456
%edx	0x124
%ecx	0x120
%ebx	
%esi	
%edi	
%esp	
%ebp	0x104

	Offset		Endereço
	-4		0x100
%ebp →			0x104
	4		0x108
xp	8	0x124	0x10c
yp	12	0x120	0x110
			0x114
			0x118
			0x11c
		456	0x120
		123	0x124

← 4 →
células

Exemplo de utilização de modos simples de endereçamento à memória no IA-32 (7)



%eax	456
%edx	0x124
%ecx	0x120
%ebx	123
%esi	
%edi	
%esp	
%ebp	0x104

Corpo

```

movl 12(%ebp), %ecx # ecx = yp
movl 8(%ebp), %edx  # edx = xp
movl (%ecx), %eax   # eax = *yp (t1)
movl (%edx), %ebx   # ebx = *xp (t0)
movl %eax, (%edx)   # *xp = eax
movl %ebx, (%ecx)   # *yp = ebx
    
```

	Offset		Endereço
	-4		0x100
%ebp →			0x104
	4		0x108
xp	8	0x124	0x10c
yp	12	0x120	0x110
			0x114
			0x118
			0x11c
		456	0x120
		123	0x124

← 4 →
células

Exemplo de utilização de modos simples de endereçamento à memória no IA-32 (8)



%eax	456
%edx	0x124
%ecx	0x120
%ebx	123
%esi	
%edi	
%esp	
%ebp	0x104

Corpo

```

movl 12(%ebp), %ecx # ecx = yp
movl 8(%ebp), %edx  # edx = xp
movl (%ecx), %eax   # eax = *yp (t1)
movl (%edx), %ebx   # ebx = *xp (t0)
movl %eax, (%edx)   # *xp = eax
movl %ebx, (%ecx)   # *yp = ebx
    
```

	Offset		Endereço
	-4		0x100
%ebp →			0x104
	4		0x108
xp	8	0x124	0x10c
yp	12	0x120	0x110
			0x114
			0x118
			0x11c
		456	0x120
		456	0x124

← 4 →
células

Exemplo de utilização de modos simples de endereçamento à memória no IA-32 (9)



Corpo

```

movl 12(%ebp), %ecx # ecx = yp
movl 8(%ebp), %edx  # edx = xp
movl (%ecx), %eax   # eax = *yp (t1)
movl (%edx), %ebx   # ebx = *xp (t0)
movl %eax, (%edx)   # *xp = eax
movl %ebx, (%ecx)   # *yp = ebx
    
```

%eax	456
%edx	0x124
%ecx	0x120
%ebx	123
%esi	
%edi	
%esp	
%ebp	0x104

	Offset		Endereço
	-4		0x100
%ebp →			0x104
	4		0x108
xp	8	0x124	0x10c
yp	12	0x120	0x110
			0x114
			0x118
			0x11c
		123	0x120
		456	0x124

← 4 →
células

Modos de endereçamento à memória no IA-32 (2)



- Indirecto (R) $\text{Mem}[\text{Reg}[R]] \dots$
- Deslocamento D(R) $\text{Mem}[\text{Reg}[R] + D] \dots$
- Indexado **D(Rb,Ri,S)** $\text{Mem}[\text{Reg}[Rb] + S * \text{Reg}[Ri] + D]$

D: Deslocamento constante de 1, 2, ou 4 *bytes*

Rb: Registo base: quaisquer dos 8 Reg Int

Ri: Registo indexação: qualquer, exceto %esp

S: Scale: 1, 2, 4, ou 8

Casos particulares:

(Rb,Ri) $\text{Mem}[\text{Reg}[Rb] + \text{Reg}[Ri]]$

D(Rb,Ri) $\text{Mem}[\text{Reg}[Rb] + \text{Reg}[Ri] + D]$

(Rb,Ri,S) $\text{Mem}[\text{Reg}[Rb] + S * \text{Reg}[Ri]]$

Exemplo de instrução do IA-32 apenas para cálculo do apontador para um operando (1)



leal Src, Dest

- **Src** contém a expressão para cálculo do endereço
- **Dest** vai receber o resultado do cálculo da expressão
- nota: lea => load effective address

- **Tipos de utilização desta instrução:**

- cálculo de um endereço de memória (sem aceder à memória)

- Ex.: tradução de `p = &x[i];`

- cálculo de expressões aritméticas do tipo

$a = c^{te} + x + k * y$ onde $c^{te} \Rightarrow$ inteiro com sinal

D(Rb, Ri, S)

x e $y \Rightarrow$ variáveis em registos

$k = 1, 2, 4, \text{ ou } 8$

- **Exemplos ...**

Exemplo de instrução do IA-32 apenas para cálculo do apontador para um operando (2)



leal Source, %eax

%edx	0xf000
------	--------

%ecx	0x100
------	-------

Source	Expressão	-> %eax
0x8 (%edx)	0xf000 + 0x8	0xf008
(%edx, %ecx)	0xf000 + 0x100	0xf100
(%edx, %ecx, 4)	0xf000 + 4*0x100	0xf400
0x80 (, %edx, 2)	2*0xf000 + 0x80	0x1e080

D(Rb, Ri, S)

Instruções de transferência de informação no IA-32



movx	S, D	$D \leftarrow S$	Move (b yte, w ord, l ong-word)
movsbl	S, D	$D \leftarrow \text{SignExtend}(S)$	Move Sign-Extended Byte
movzbl	S, D	$D \leftarrow \text{ZeroExtend}(S)$	Move Zero-Extended Byte
push	S	$\%esp \leftarrow \%esp - 4; \text{Mem}[\%esp] \leftarrow S$	Push
pop	D	$D \leftarrow \text{Mem}[\%esp]; \%esp \leftarrow \%esp + 4$	Pop
lea	S, D	$D \leftarrow \&S$	Load Effective Address

D – destino [Reg | Mem]

S – fonte [Imm | Reg | Mem]

D e **S** não podem ser ambos operandos em memória no IA-32

Operações aritméticas e lógicas no IA-32



inc D	$D \leftarrow D + 1$	Increment
dec D	$D \leftarrow D - 1$	Decrement
neg D	$D \leftarrow -D$	Negate
not D	$D \leftarrow \sim D$	Complement
add S, D	$D \leftarrow D + S$	Add
sub S, D	$D \leftarrow D - S$	Subtract
imul S, D	$D \leftarrow D * S$	32 bit Multiply
and S, D	$D \leftarrow D \& S$	And
or S, D	$D \leftarrow D S$	Or
xor S, D	$D \leftarrow D \wedge S$	Exclusive-Or
shl k, D	$D \leftarrow D \ll k$	Left Shift
sar k, D	$D \leftarrow D \gg k$	Arithmetic Right Shift
shr k, D	$D \leftarrow D \gg k$	Logical Right Shift

Análise do Instruction Set Architecture (3)



Estrutura do tema ISA do IA-32

1. Desenvolvimento de programas no IA-32 em Linux
2. Acesso a operandos e operações
- 3. Suporte a estruturas de controlo**
4. Suporte à invocação/regresso de funções
5. Análise comparativa: IA-32 vs. x86-64 e RISC (MIPS e ARM)
6. Acesso e manipulação de dados estruturados

Alteração do fluxo de execução de instruções



- **Por omissão, as instruções são sempre executadas sequencialmente, i.e., uma após outra** (em HLL & em ling. máq.)
- **Em HLL o fluxo de instruções poderá ser alterado:**
 - na execução de estruturas de controlo (*nestes slides...*)
 - na invocação / regresso de funções (*mais adiante...*)
 - na ocorrência de exceções / interrupções (*mais adiante?*)
- **Em ling. máq. isso traduz-se na alteração do IP, de modo incondicional / condicional, por um valor absoluto / relativo**
 - **jump / branch / skip** (no IA-32 apenas **jmp**)
 - **call** (com salvaguarda do endereço de regresso) e **ret**
 - em exceções / interrupções . . .

Instruções de controlo de fluxo no IA-32



`jmp Label    %eip  $\leftarrow$  Label` Unconditional jump

`je Label` Jump if Zero/Equal

`js Label` Jump if Negative

`jg Label` Jump if Greater (signed $>$)

`jge Label` Jump if Greater or equal (signed $\geq$)

`ja Label` Jump if Above (unsigned $>$)

`jb Label` Jump if Below (unsigned $<$)

`call Label    pushl %eip; %eip  $\leftarrow$  Label` Procedure call

`ret` popl %eip Procedure return

Estruturas de controlo de uma linguagem imperativa



Estruturas de controlo em C

– *if-else statement*

Estrutura geral:

```
...  
    if (condição)  
        expressão_1;  
    else  
        expressão_2;  
...
```

Exemplo:

```
int absdiff(int x, int y)  
{  
    if (x < y)  
        return y - x;  
    else  
        return x - y;  
}
```

– *do-while statement*

– *while statement*

– *for loop*

– *switch statement*

Assembly:

"condição":

expressão Booleana, V ou F

"if condição statement":

salte se V para ...

Codificação das condições no IA-32

(para utilização posterior)



- **Condições (V/F) codificadas a partir de registos de 1 bit -> *Flags***

ZF *Zero Flag* SF *Sign Flag*
OF *Overflow Flag* CF *Carry Flag*

- **As *Flags* podem ser implícita ou explicitamente alteradas:**

- implicitamente, por operações aritméticas/lógicas; *exemplo:*

addl *Src, Dest* **Equivalente em C:** $a = a + b$
Flags afetadas: ZF SF OF CF

- explicitamente, por instruções de comparação e teste

cmpl *Src2, Src1* **Equivalente em C...** apenas calcula $Src1 - Src2$
Flags afetadas: ZF SF OF CF
testl *Src2, Src1* **Equivalente em C...** apenas calcula $Src1 \& Src2$
Flags afetadas: ZF SF OF CF

Utilização das Flags no IA-32



A condição codificada a partir das *Flags* pode ser:

- Colocada diretamente num registo de 8 bits (V/F) **ou...**

`setcc Dest` *Dest:* %al %ah %dl %dh %ch %cl %bh %bl

Nota: não altera restantes 3 bytes do reg de 32 bits; usada normal/ com `movzbl`

- Usada numa instrução de salto condicional:

`jcc Label` *Label:* endereço destino ou distância para destino

Códigos de condição (cc):

<code>(set/j) cc</code>	Descrição	Flags
<code>(set/j) e, z</code>	<i>Equal, zero</i>	ZF
<code>(set/j) ne</code>	<i>Not Equal</i>	~ZF
<code>(set/j) s</code>	<i>Sign (-)</i>	SF
<code>(set/j) ns</code>	<i>Not Sign (-)</i>	~SF

<code>(set/j) g</code>	$> (c/ sinal)$	$\sim(SF \wedge OF) \& \sim ZF$
<code>(set/j) ge</code>	$\geq (c/ sinal)$	$\sim(SF \wedge OF)$
<code>(set/j) l</code>	$< (c/ sinal)$	$(SF \wedge OF)$
<code>(set/j) le</code>	$\leq (c/ sinal)$	$(SF \wedge OF) ZF$
<code>(set/j) a</code>	$> (s/ sinal)$	$\sim CF \& \sim ZF$
<code>(set/j) b</code>	$< (s/ sinal)$	CF

if-then-else statement (1)



Análise de um exemplo

```
int absdiff(int x, int y)
{
    if (x < y)
        return y - x;
    else
        return x - y;
}
```

C original

Corpo {

```
    movl 8(%ebp), %edx
    movl 12(%ebp), %eax
    cmpl %eax, %edx
    jl   .L3
    subl %eax, %edx
    movl %edx, %eax
    jmp  .L5
.L3:
    subl %edx, %eax
.L5:
```

```
int goto_diff(int x, int y)
{
    int rval;
    if (x < y)
        goto then_statement;
    rval = x - y;
    goto done;
then_statement:
    rval = y - x;
done:
    return rval;
}
```

Versão goto

```
# edx = x
# eax = y
# compare x : y (~ x-y)
# if x < y, goto then_statement
# compute x - y
# return the value (x - y)
# goto done
# then_statement:
# return the value (y - x)
# done:
```

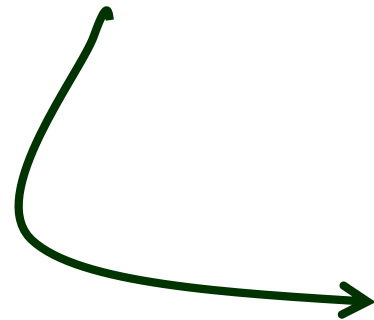
if-then-else statement (2)



Generalização

```
if (expressão_de_teste)  
    then_statement  
else  
    else_statement
```

Forma genérica em C



```
cond = expressão_de_teste  
if (cond)  
    goto true;  
else_statement  
goto done;  
true:  
    then_statement  
done:
```

**Versão com *goto*, ou
assembly com sintaxe C**

if-then-else statement (3)



Generalização alternativa

```
if (expressão_de_teste)  
    then_statement  
else  
    else_statement
```

Forma genérica em C

```
cond = expressão_de_teste  
if (~cond)  
    goto else;  
then_statement  
goto done;  
else:  
    else_statement  
done:
```

**Versão com *goto*, ou
assembly com sintaxe C**

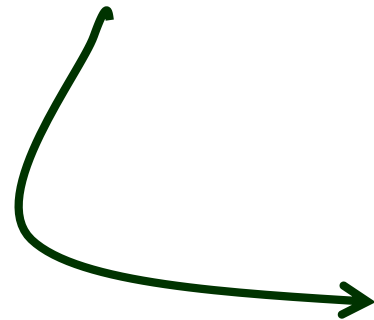
if-then-else statement (4)



Generalização alternativa (*sem else*)

```
if (expressão_de_teste)
    then_statement
else
    else_statement
```

Forma genérica em C



```
cond = expressão_de_teste
if (~cond)
    goto done;
then_statement
goto done;
else:
    else_statement
done:
```

**Versão com *goto*, ou
assembly com sintaxe C**

do-while statement (1)



Generalização

```
do  
    body_statement  
while (expressão_de_teste) ;
```

Forma genérica em C

```
loop:  
    body_statement  
    cond = expressão_de_teste  
    if (cond)  
        goto loop;
```

Versão com *goto*, ou
assembly com sintaxe C

do-while statement (2)



Análise de um exemplo

– série de Fibonacci:

$$F_1 = F_2 = 1$$

$$F_n = F_{n-1} + F_{n-2}, \quad n \geq 3$$

```
int fib_dw(int n)
{
    int i = 0;
    int val = 0;
    int nval = 1;

    do {
        int t = val + nval;
        val = nval;
        nval = t;
        i++;
    } while (i < n);

    return val;
}
```

C original

```
int fib_dw_goto(int n)
{
    int i = 0;
    int val = 0;
    int nval = 1;

loop:
    int t = val + nval;
    val = nval;
    nval = t;
    i++;
    if (i < n);
        goto loop;
    return val;
}
```

Versão com goto

do-while statement (3)



Análise de um exemplo – série de Fibonacci

Utilização dos registos		
Variável	Registo	Valor inicial
n	%esi	n (argumento)
i	%ecx	0
val	%ebx	0
nval	%edx	1
t	%eax	1

Corpo
(loop)

.L2:

```
leal (%edx,%ebx),%eax
movl %edx,%ebx
movl %eax,%edx
incl %ecx
cmpl %esi,%ecx
jl .L2
movl %ebx,%eax
```

```
int fib_dw_goto(int n)
{
    int i = 0;
    int val = 0;
    int nval = 1;

loop:
    int t = val + nval;
    val = nval;
    nval = t;
    i++;
    if (i<n);
        goto loop;
    return val;
}
```

Versão goto

```
# loop:
# t = val + nval
# val = nval
# nval = t
# i++
# compare i : n
# if i<n, goto loop
# para devolver val
```

while statement (1)



Generalização

```
while (expressão_de_teste)  
    body_statement
```

Forma genérica em C

```
loop:  
    cond = expressão_de_teste  
    if (! cond)  
        goto done;  
    body_statement  
    goto loop;  
done:
```

Versão com *goto*

```
if (! expressão_de_teste)  
    goto done;  
do  
    body_statement  
    while (expressão_de_teste) ;  
done:
```

Conversão *while* em *do-while*

```
cond = expressão_de_teste  
if (! cond)  
    goto done;  
loop:  
    body_statement  
    cond = expressão_de_teste  
    if (cond)  
        goto loop;  
done:
```

Versão *do-while* com *goto*

while statement (2)



Análise de um exemplo

– série de Fibonacci

```
int fib_w(int n)
{
    int i = 1;
    int val = 1;
    int nval = 1;

    while (i < n) {
        int t = val + nval;
        val = nval;
        nval = t;
        i++;
    }

    return val;
}
```

C original

```
int fib_w_goto(int n)
{
    int i = 1;
    int val = 1;
    int nval = 1;

    if (i ≥ n);
        goto done;

loop:
    int t = val + nval;
    val = nval;
    nval = t;
    i++;
    if (i < n);
        goto loop;
done:
    return val;
}
```

Versão do-while com goto

while statement (3)



Análise de um exemplo – série de Fibonacci

Utilização dos registos		
Variável	Registo	Valor inicial
n	%esi	n
i	%ecx	1
val	%ebx	1
nval	%edx	1
t	%eax	2

```
int fib_w_goto(int n)
{
    (...)

    if (i ≥ n);
        goto done;

loop:
    (...)
    if (i < n);
        goto loop;
done:
    return val;
}
```

**Versão
do-while
com goto**

Corpo {

```
(...)
    cmp1 %esi,%ecx
    jge .L7
.L5:
    (...)
    cmp1 %esi,%ecx
    jl .L5
.L7:
    mov1 %ebx,%eax
```

esi=n, i=val=nval=1
compare i : n
if i ≥ n, goto done
loop:
compare i : n
if i < n, goto loop
done:
return val

**Nota: Código
gerado com
gcc -O1 -S**

for loop (1)



Generalização

```
for (expr_inic; expr_test; update)  
  body_statement
```

Forma genérica em C

```
expr_inic ;  
while (expr_test) {  
  body_statement  
  update ;  
}
```

Conversão
for em
while

```
expr_inic ;  
if (! expr_test)  
  goto done;  
do {  
  body_statement  
  update ;  
} while (expr_test) ;  
done:
```

Conversão
para
do-while

```
expr_inic ;  
cond = expr_test ;  
if (! cond)  
  goto done;  
loop:  
  body_statement  
  update ;  
  cond = expr_test ;  
  if (cond)  
    goto loop;  
done:
```

Versão
do-while
com goto

for loop (2)



Análise de um exemplo

– série de Fibonacci

```
int fib_f(int n)
{
    int i;
    int val = 1;
    int nval = 1;

    for (i=1; i<n; i++) {
        int t = val + nval;
        val = nval;
        nval = t;
    }

    return val;
}
```

C original

```
int fib_f_goto(int n)
{
    int val = 1;
    int nval = 1;

    int i = 1;
    if (i >= n)
        goto done;

loop:
    int t = val + nval;
    val = nval;
    nval = t;
    i++;
    if (i < n)
        goto loop;
done:
    return val;
}
```

Versão do-while com goto

Nota: gcc gera mesmo código...

switch statement



**"Salto" com escolha múltipla;
alternativas de implementação:**

- Sequência de `if-then-else` *statements*
- Com saltos "indiretos": endereços especificados numa tabela de salto (*jump table*)

Análise duma Instruction Set Architecture (4)



Estrutura do tema ISA do IA-32

1. Desenvolvimento de programas no IA-32 em Linux
2. Acesso a operandos e operações
3. Suporte a estruturas de controlo
- 4. Suporte à invocação/regresso de funções**
5. Análise comparativa: IA-32 vs. x86-64 e RISC (MIPS e ARM)
6. Acesso e manipulação de dados estruturados

Suporte a funções e procedimentos no IA-32 (1)



Estrutura de uma função (/ procedimento)

- **função versus procedimento** (ou ainda **rotina**, em Fortran)
 - o nome duma função é usado como se fosse uma variável
 - uma função devolve um valor, um procedimento não
- **parte visível ao programador em HLL**
 - o código do corpo da função
 - a passagem de parâmetros/argumentos para a função ...
... e o valor devolvido pela função
 - o alcance das variáveis: locais, externas ou globais
- **parte não visível em HLL** (gestão do contexto da função)
 - variáveis locais (propriedades)
 - variáveis externas e globais (localização e acesso)
 - parâms / argum's e valor a devolver pela função (propriedades)
 - gestão do contexto (controlo & dados)

Suporte a funções e procedimentos no IA-32 (2)



Análise do contexto de uma função

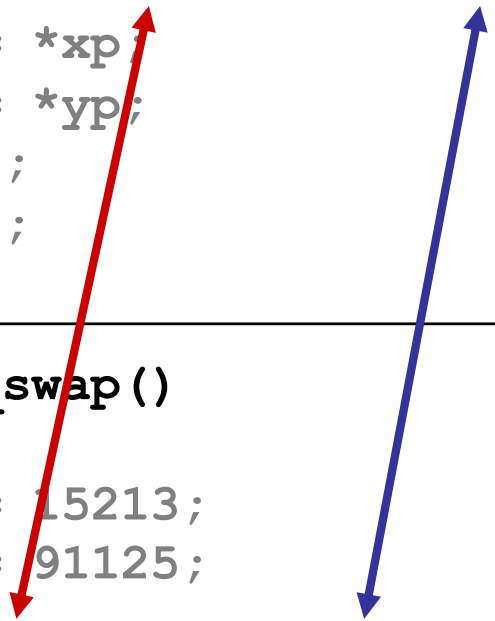
- **propriedades das variáveis locais:**
 - visíveis apenas durante a execução da função
 - deve suportar aninhamento e recursividade
 - localização ideal (escalares): em registo, se os houver...
 - localização no código em IA-32: em registo, enquanto houver...
- **variáveis externas e globais:**
 - externas: valor ou localização expressa na lista de argumentos
 - globais: localização definida pelo *linker & loader* (IA-32: na memória)
- **propriedades dos parâmetros / arg's (só de entrada em C):**
 - por valor (c^{te} ou valor da variável) ou por referência (localização da variável)
 - designação independente (f. chamadora / f. chamada): →
 - deve ...
 - localização ideal: ...
 - localização no código em IA-32: ...
- **valor a devolver pela função:**
 - é ...
 - localização: ...
- **gestão do contexto ...**

Designação independente dos parâmetros



```
void swap(int *xp, int *yp)
{
    int t0 = *xp;
    int t1 = *yp;
    *xp = t1;
    *yp = t0;
}
```

```
void call_swap()
{
    int zip1 = 15213;
    int zip2 = 91125;
    (...)
    swap(&zip1, &zip2);
    (...)
}
```



Suporte a funções e procedimentos no IA-32 (2)



Análise do contexto de uma função

- **propriedades das variáveis locais:**
 - visíveis apenas durante a execução da função
 - deve suportar aninhamento e recursividade
 - localização ideal (escalares): em registo, se os houver...
 - localização no código em IA-32: em registo, enquanto houver...
- **variáveis externas e globais:**
 - externas: valor ou localização expressa na lista de argumentos
 - globais: localização definida pelo *linker* & *loader* (IA-32: na memória)
- **propriedades dos parâmetros/arg's** (só de entrada em C):
 - por valor (c^{te} ou valor da variável) ou por referência (localização da variável)
 - designação independente (f. chamadora / f. chamada)
 - deve suportar aninhamento e recursividade
 - localização ideal: em registo, se os houver; mas...
 - localização no código em IA-32: na memória (na *stack*)
- **valor a devolver pela função:**
 - é uma quantidade escalar, do tipo inteiro, real ou apontador
 - localização: em registo (IA-32: `int` no registo `eax` e/ou `edx`)
- **gestão do contexto** (controlo & dados) ...

Suporte a funções e procedimentos no IA-32 (3)



Análise do código de gestão de uma função

- **invocação e regresso**
 - instrução de salto, c/ salvaguarda IP (endereço de regresso)
 - em registo (RISC; aninhamento / recursividade ?)
 - em memória/na *stack* (IA-32; aninhamento / recursividade ?)
- **invocação e regresso**
 - instrução de salto para o endereço de regresso
- **salvaguarda & recuperação de registos** (na *stack*)
 - função chamadora ? (nenhum/ alguns/ todos ? RISC/IA-32 ?)
 - função chamada? (nenhum/ alguns/ todos ? RISC/IA-32 ?)
- **gestão do contexto ...**

Utilização de registos em funções: regras seguidas pelos compiladores para IA-32



Utilização dos registos (de inteiros)

–Três do tipo *caller-save*

%eax, %edx, %ecx

- *save/restore*: função chamadora

–Três do tipo *callee-save*

%ebx, %esi, %edi

- *save/restore*: função chamada

–Dois apontadores (para a *stack*)

%esp, %ebp

- topo da *stack*, base/referência na *stack*

Caller-Save

Callee-Save

Pointers

%eax

%edx

%ecx

%ebx

%esi

%edi

%esp

%ebp

Nota: valor a devolver pela função vai em **%eax**

Suporte a funções e procedimentos no IA-32 (3)



Análise do código de gestão de uma função

- invocação e regresso
 - instrução de salto, c/ salvaguarda IP (endereço de regresso)
 - em registo (RISC; aninhamento / recursividade ?)
 - em memória/na *stack* (IA-32; aninhamento / recursividade ?)
- invocação e regresso
 - instrução de salto para o endereço de regresso
- salvaguarda & recuperação de registos (na *stack*)
 - função chamadora ? (nenhum/ alguns/ todos ? RISC/IA-32 ?)
 - função chamada? (nenhum/ alguns/ todos ? RISC/IA-32 ?)
- **gestão do contexto** (em stack frame ou *activation record*)
 - reserva/libertação de espaço para variáveis locais
 - atualização/recuperação do *frame pointer* (IA-32...)

Suporte a funções e procedimentos no IA-32 (4)



Análise de exemplos

– revisão do exemplo swap

- análise das fases: arranque/inicialização, corpo, término
- análise dos contextos (IA-32)
- evolução dos contextos na *stack* (IA-32)

– evolução de um exemplo: Fibonacci

- análise ...

– aninhamento e recursividade

- evolução ...

Análise das fases em swap, no IA-32 (fig. já apresentada)



```
void swap(int *xp, int *yp)
{
    int t0 = *xp;
    int t1 = *yp;
    *xp = t1;
    *yp = t0;
}
```

Código C

swap:

```
    pushl %ebp
    movl  %esp,%ebp
    pushl %ebx
```

Arranque

```
    movl 12(%ebp),%ecx
    movl 8(%ebp),%edx
    movl (%ecx),%eax
    movl (%edx),%ebx
    movl %eax, (%edx)
    movl %ebx, (%ecx)
```

Corpo

```
    movl -4(%ebp),%ebx
    movl %ebp,%esp
    popl %ebp
    ret
```

Término

Assembly

Análise dos contextos em swap, no IA-32



```
void swap(int *xp, int *yp)
{
    int t0 = *xp;
    int t1 = *yp;
    *xp = t1;
    *yp = t0;
}
```

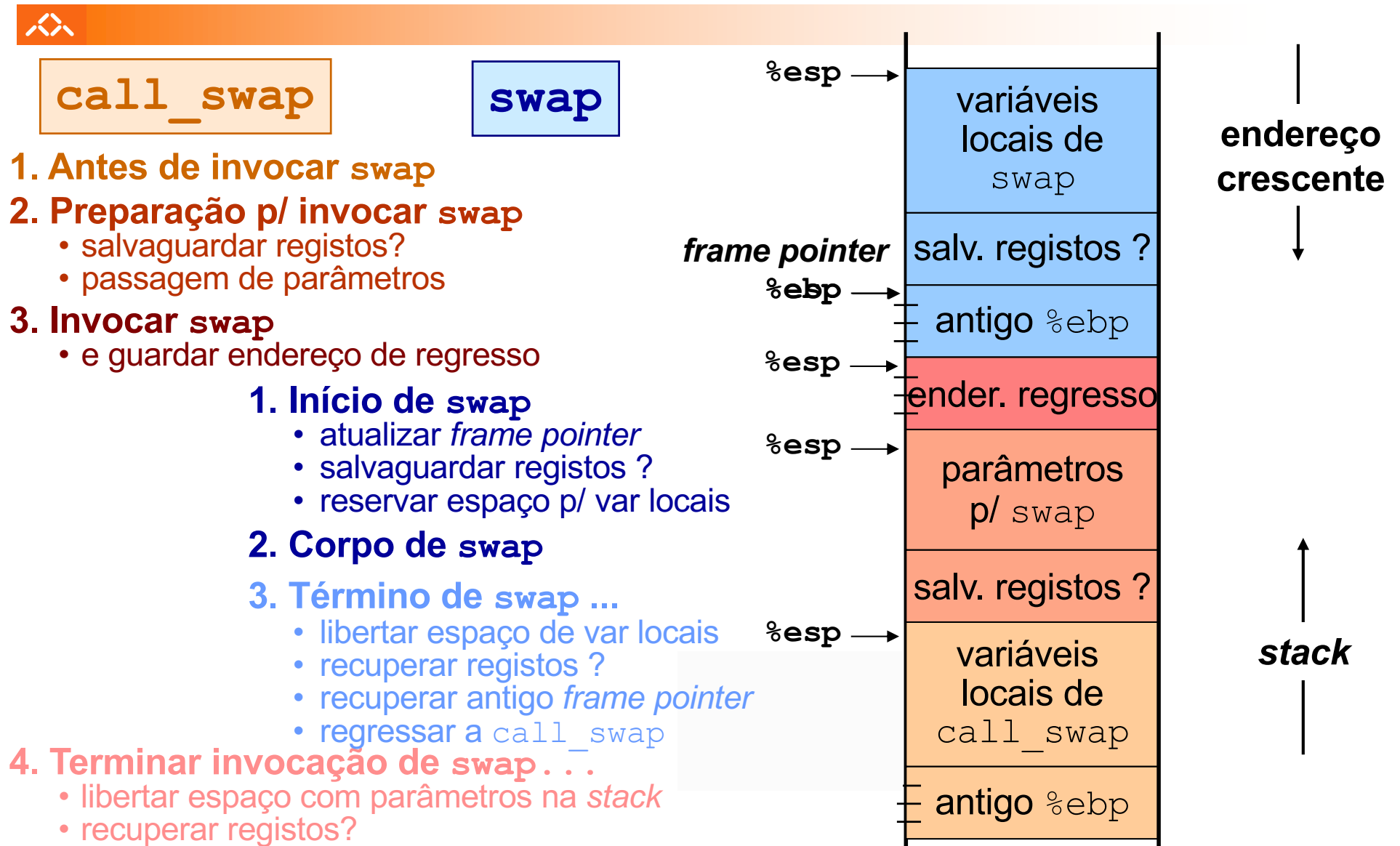
```
void call_swap()
{
    int zip1 = 15213;
    int zip2 = 91125;
    (...)
    swap(&zip1, &zip2);
    (...)
}
```

- em `call_swap`
- na invocação de `swap`
- na execução de `swap`
- no regresso a `call_swap`

Que contextos (IA-32)?

- passagem de parâmetros
 - via *stack*
- espaço para variáveis locais
 - na *stack*
- info de suporte à gestão (*stack*)
 - endereço de regresso
 - apontador para a *stack frame*
 - salvaguarda de registos

Construção do contexto na stack, no IA-32



Evolução da stack, no IA-32 (1)

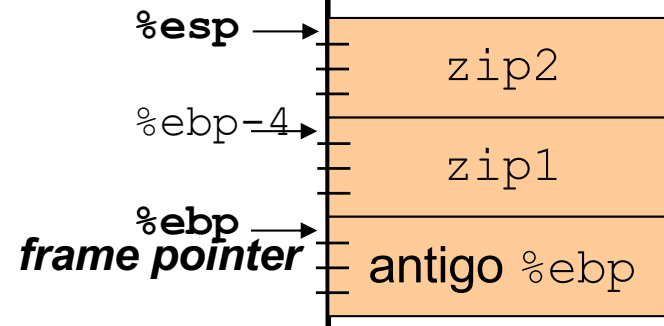


call_swap

1. Antes de invocar swap

```
void swap(int *xp, int *yp)
{
    int t0 = *xp;
    int t1 = *yp;
    *xp = t1;
    *yp = t0;
}
```

```
void call_swap()
{
    int zip1 = 15213;
    int zip2 = 91125;
    (...)
    swap(&zip1, &zip2);
    (...)
}
```



endereço
crescente
↓

↑
stack

Evolução da stack, no IA-32 (2)



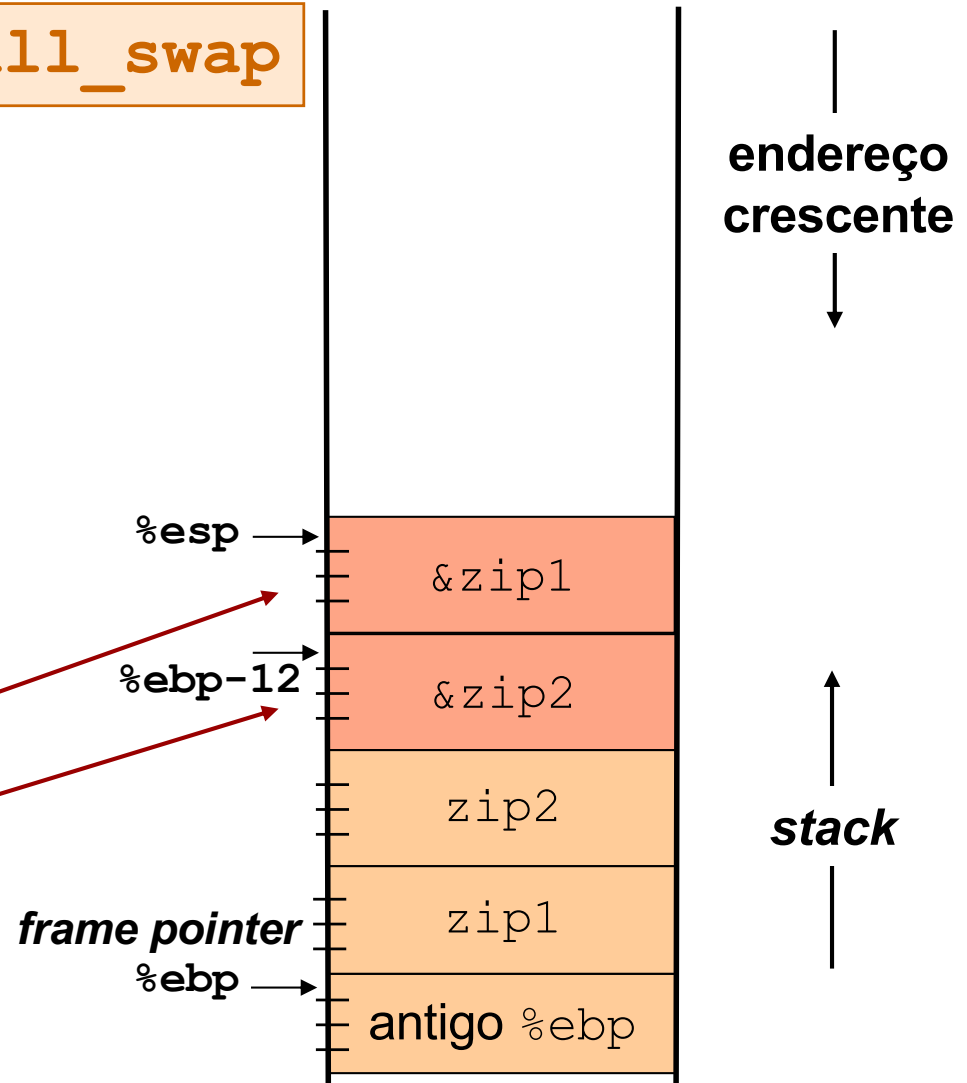
2. Preparação p/ invocar swap

- salvar registos?...**não**...
- passagem de parâmetros

call_swap

```
void swap(int *xp, int *yp)
{
    int t0 = *xp;
    int t1 = *yp;
    *xp = t1;
    *yp = t0;
}
```

```
void call_swap()
{
    int zip1 = 15213;
    int zip2 = 91125;
    (...)
    swap(&zip1, &zip2);
    (...)
}
```



Evolução da stack, no IA-32 (3)

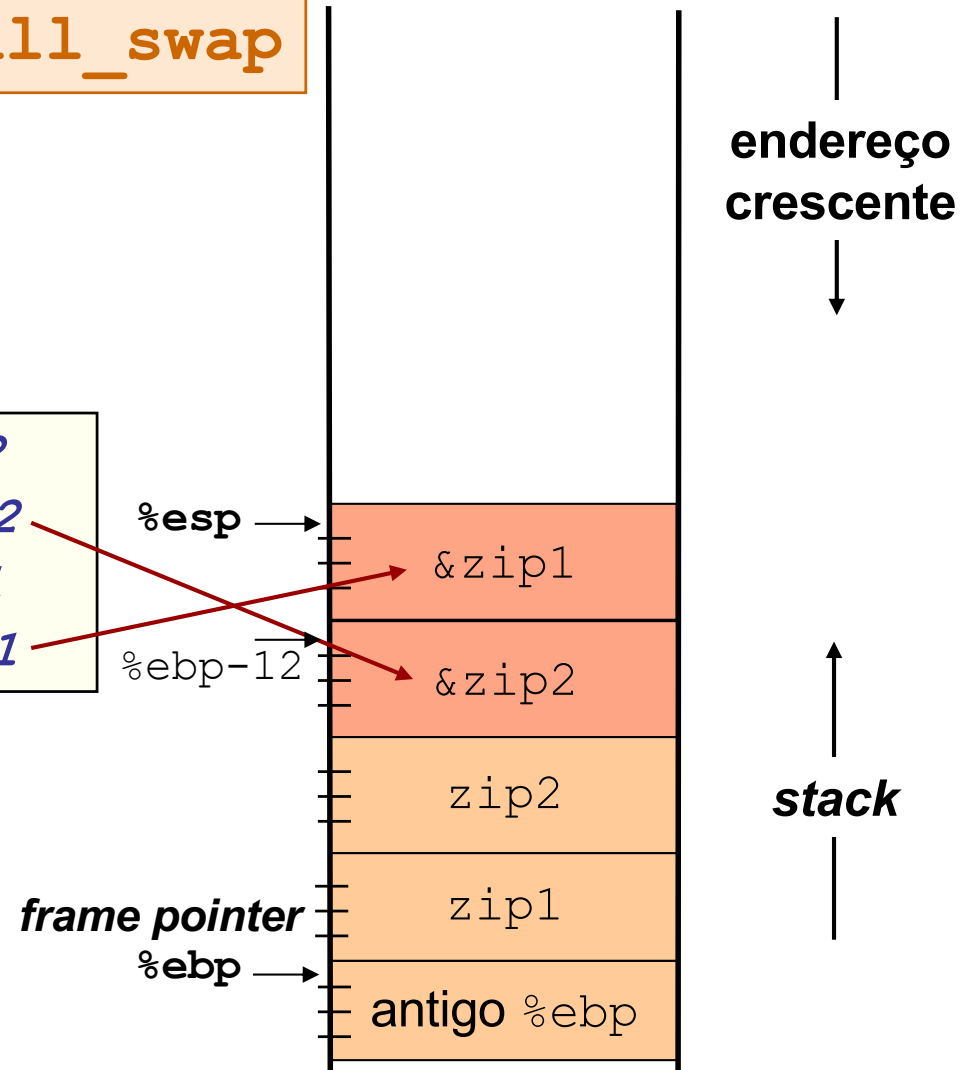


call_swap

2. Preparação p/ invocar swap

- salvar registos?...**não**...
- passagem de parâmetros

```
leal  -8(%ebp), %eax  Calcula &zip2  
pushl %eax            Empilha &zip2  
leal  -4(%ebp), %eax  Calcula &zip1  
pushl %eax            Empilha &zip1
```



Evolução da stack, no IA-32 (4)



3. Invocar swap

- e guardar endereço de regresso

```
void call_swap()  
{  
  int zip1 = 15213;  
  int zip2 = 91125;  
  (...)  
  swap(&zip1, &zip2);  
  (...)  
}
```

call swap

Invoca função swap

call_swap

frame pointer
%ebp →

%esp →

%ebp-12 →

antigo %eip

&zip1

&zip2

zip2

zip1

antigo %ebp

endereço
crescente
↓

↑
stack

Evolução da stack, no IA-32 (5)



1. Início de swap

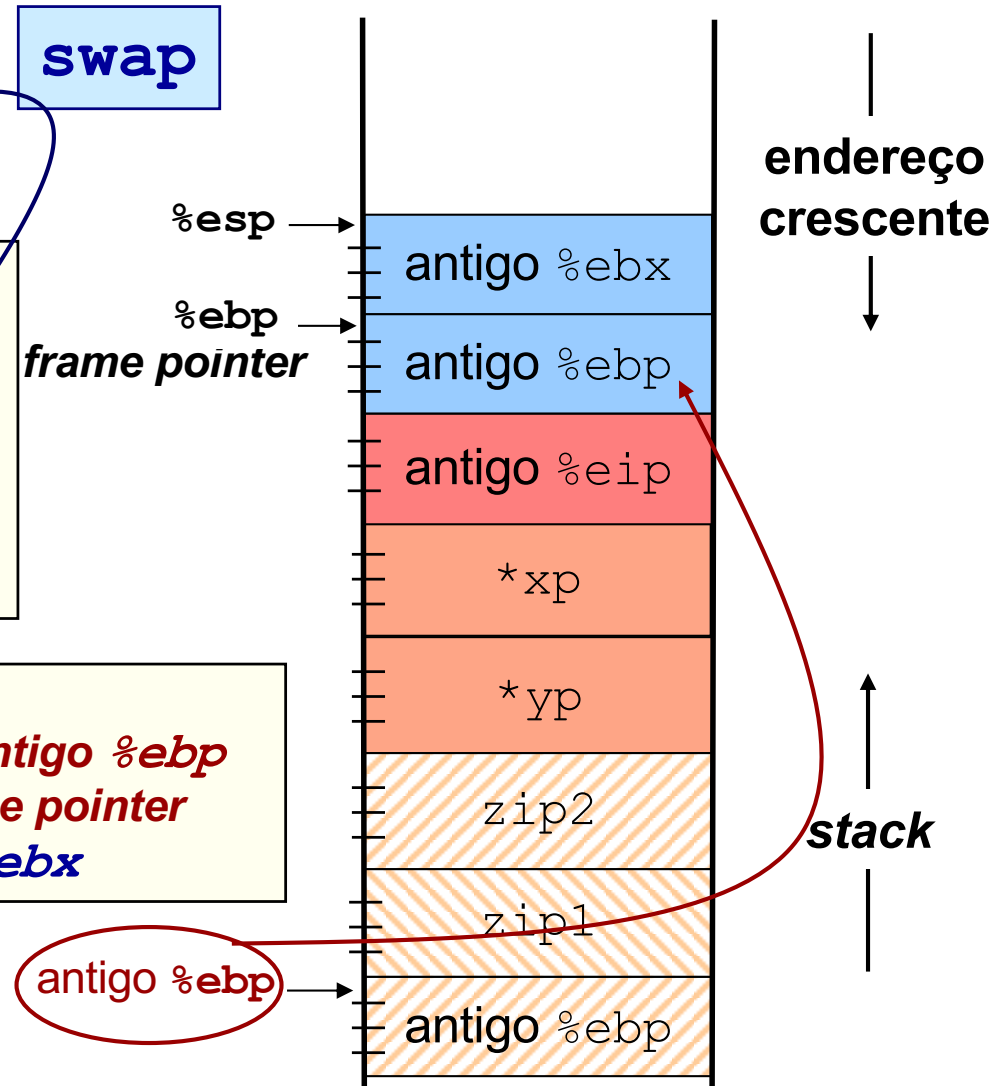
- atualizar *frame pointer*
- salvar registos
- reservar espaço p/ locais...*não...*

```
void swap(int *xp, int *yp)
{
    int t0 = *xp;
    int t1 = *yp;
    *xp = t1;
    *yp = t0;
}
```

```
swap:
    pushl    %ebp
    movl     %esp, %ebp
    pushl    %ebx
```

Salvaguarda antigo %ebp
Faz %ebp frame pointer
Salvaguarda %ebx

swap



Evolução da stack, no IA-32 (6)

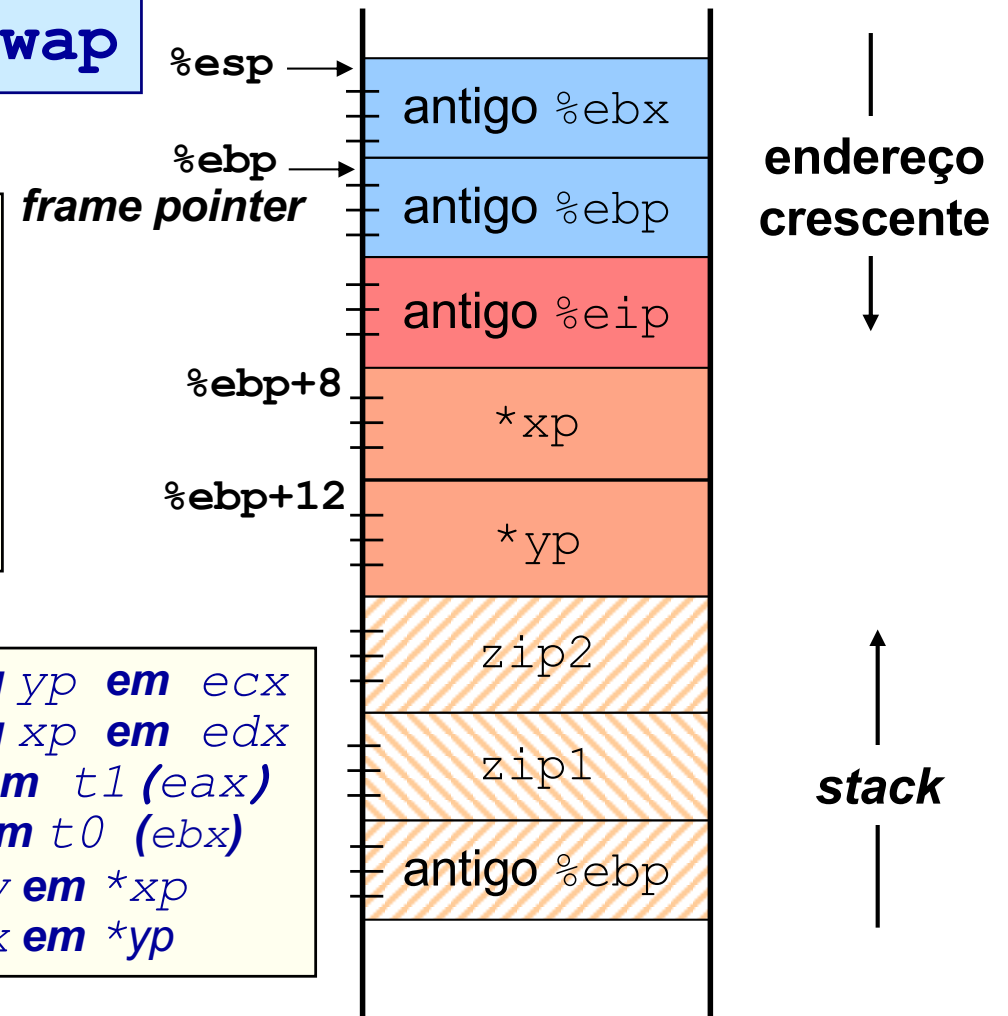


2. Corpo de swap

```
void swap(int *xp, int *yp)
{
    int t0 = *xp;
    int t1 = *yp;
    *xp = t1;
    *yp = t0;
}
```

<code>movl 12(%ebp), %ecx</code>	Carrega arg yp em ecx
<code>movl 8(%ebp), %edx</code>	Carrega arg xp em edx
<code>movl (%ecx), %eax</code>	Coloca y em t1 (eax)
<code>movl (%edx), %ebx</code>	Coloca x em t0 (ebx)
<code>movl %eax, (%edx)</code>	Armazena y em *xp
<code>movl %ebx, (%ecx)</code>	Armazena x em *yp

swap



Evolução da stack, no IA-32 (7)



3. Término de swap ...

- libertar espaço de var locais...**não...**
- recuperar registos
- recuperar antigo frame pointer
- regressar a `call_swap`

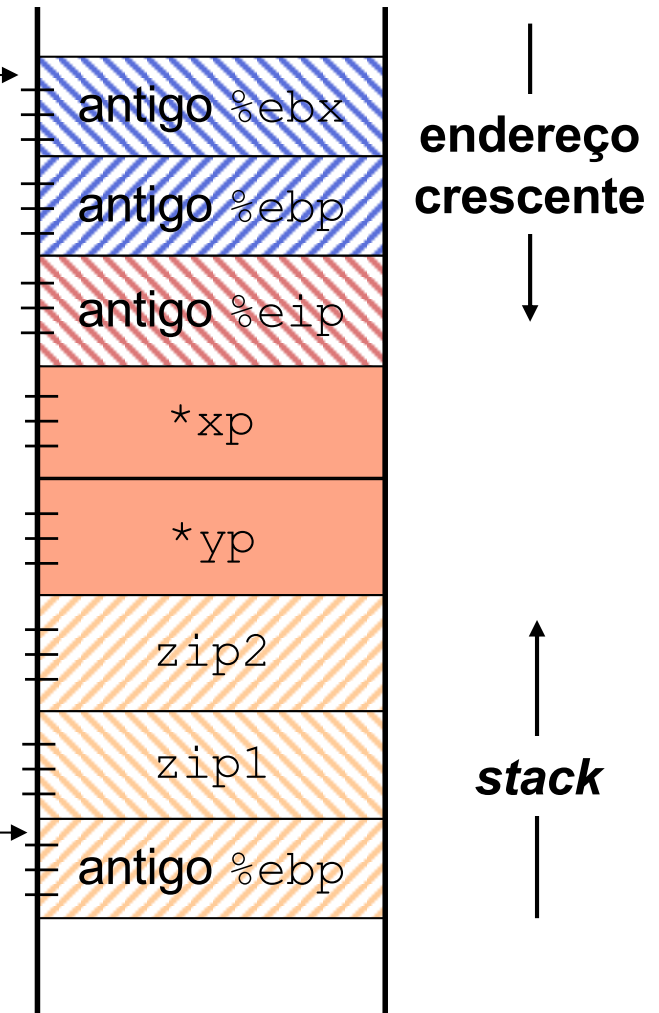
```
void swap(int *xp, int *yp)
{
    (...)
}
```

```
popl %ebx          Recupera %ebx
movl %ebp, %esp    Recupera %esp
popl %ebp          Recupera %ebp
                   ou
leave              Recupera %esp, %ebp
ret                Regressa à f. chamadora
```

swap

%esp →

%ebp →
frame
pointer



Evolução da stack, no IA-32 (8)



call_swap

4. Terminar invocação de swap...

- libertar espaço de parâmetros na *stack*...
- recuperar registos?...**não**...

```
void call_swap()
{
  int zip1 = 15213;
  int zip2 = 91125;
  (...)
  swap(&zip1, &zip2);
  (...)
}
```

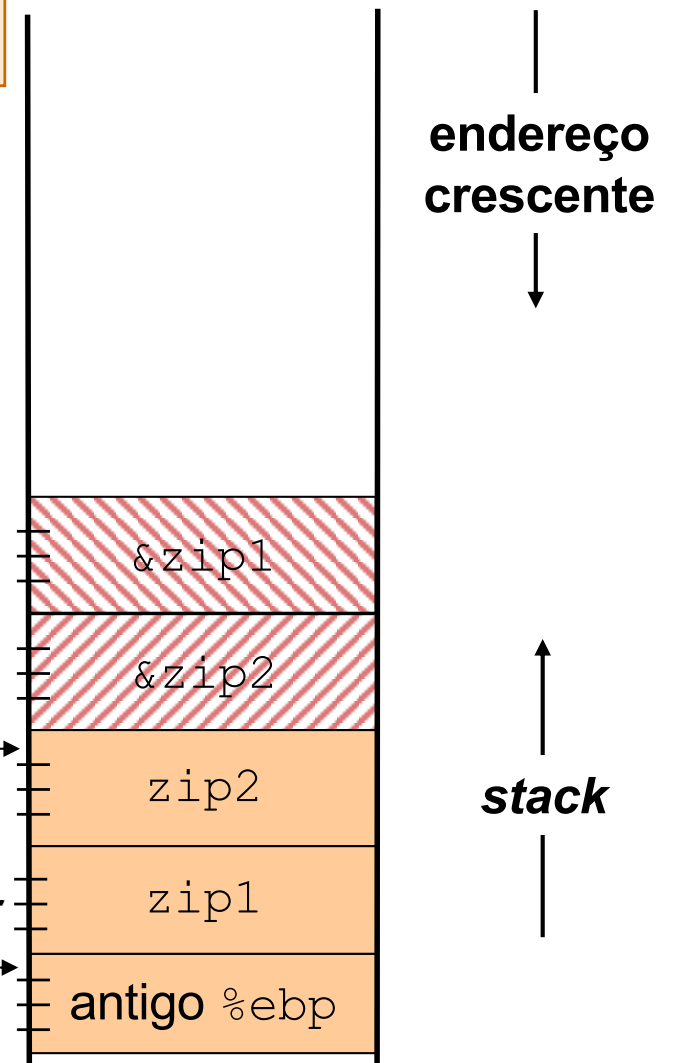
```
addl $8,%esp
```

Atualiza stack pointer

frame pointer

%ebp

%esp



Suporte a funções e procedimentos no IA-32 (4)



Análise de exemplos

- revisão do exemplo swap
 - análise das fases: arranque/inicialização, corpo, término
 - análise dos contextos (IA-32)
 - evolução dos contextos na *stack* (IA-32)
- evolução de um exemplo: Fibonacci
 - análise de uma compilação do gcc
- aninhamento e recursividade
 - evolução ...

A série de Fibonacci no IA-32 (1)

```
int fib_dw(int n)
{
    int i = 0;
    int val = 0;
    int nval = 1;

    do {
        int t = val + nval;
        val = nval;
        nval = t;
        i++;
    } while (i < n);

    return val;
}
```

do-while

```
int fib_f(int n)
{
    int i;
    int val = 1;
    int nval = 1;

    for (i=1; i<n; i++) {
        int t = val + nval;
        val = nval;
        nval = t;
    }

    return val;
}
```

for

```
int fib_w(int n)
{
    int i = 1;
    int val = 1;
    int nval = 1;

    while (i < n) {
        int t = val + nval;
        val = nval;
        nval = t;
        i++;
    }

    return val;
}
```

while

função recursiva

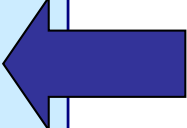
```
int fib_rec (int n)
{
    int prev_val, val;
    if (n ≤ 2)
        return (1);
    prev_val = fib_rec (n-2);
    val = fib_rec (n-1);
    return (prev_val+val);
}
```


A série de Fibonacci no IA-32 (2)



função recursiva

```
int fib_rec (int n)
{
    int prev_val, val;
    if (n ≤ 2)
        return (1);
    prev_val = fib_rec (n-2);
    val = fib_rec (n-1);
    return (prev_val+val);
}
```



_fib_rec:

```
    pushl %ebp
    movl  %esp, %ebp

    subl  $12, %esp
    movl  %ebx, -8(%ebp)
    movl  %esi, -4(%ebp)

    movl  8(%ebp), %esi
```

Atualiza frame pointer

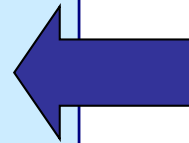
*Reserva espaço na stack para 3 int's
Salvaguarda os 2 reg's que vão ser usados;
de notar a forma de usar a stack...*

A série de Fibonacci no IA-32 (3)



função recursiva

```
int fib_rec (int n)
{
    int prev_val, val;
    if (n ≤ 2)
        return (1);
    prev_val = fib_rec (n-2);
    val = fib_rec (n-1);
    return (prev_val+val);
}
```



```
...
movl    %esi, -4(%ebp)
movl    8(%ebp), %esi
movl    $1, %eax
cmpl    $2, %esi
jle     L1
leal    -2(%esi), %eax
...
L1:
movl    -8(%ebp), %ebx
```

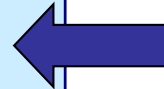
*Coloca o argumento n em $\%esi$
Coloca já o valor a devolver em $\%eax$
Compara $n:2$
Se $n \leq 2$, salta para o fim
Se não, ...*

A série de Fibonacci no IA-32 (4)



função recursiva

```
int fib_rec (int n)
{
    int prev_val, val;
    if (n ≤ 2)
        return (1);
    prev_val = fib_rec (n-2);
    val = fib_rec (n-1);
    return (prev_val+val);
}
```



```
...
jle     L1
leal    -2(%esi), %eax
movl    %eax, (%esp)
call    fib_rec
movl    %eax, %ebx
leal    -1(%esi), %eax
...
```

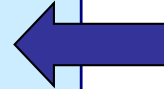
Se $n \leq 2$, salta para o fim
Se não, ... **calcula $n-2$, e...**
... **coloca-o no topo da stack (argumento)**
Invoca a função `fib_rec` e ...
... **guarda o valor de `prev_val` em `%ebx`**

A série de Fibonacci no IA-32 (5)



função recursiva

```
int fib_rec (int n)
{
    int prev_val, val;
    if (n ≤ 2)
        return (1);
    prev_val = fib_rec (n-2);
    val = fib_rec (n-1);
    return (prev_val+val);
}
```



```
...
movl  %eax, %ebx
leal  -1(%esi), %eax
movl  %eax, (%esp)
call  _fib_rec
leal  (%eax,%ebx), %eax
...
```

Calcula n-1, e...

... coloca-o no topo da stack (argumento)

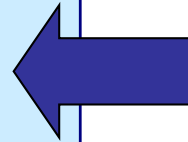
Chama de novo a função `fib_rec`

A série de Fibonacci no IA-32 (6)



função recursiva

```
int fib_rec (int n)
{
    int prev_val, val;
    if (n ≤ 2)
        return (1);
    prev_val = fib_rec (n-2);
    val = fib_rec (n-1);
    return (prev_val+val);
}
```



```
    call    fib_rec
    leal    (%eax,%ebx), %eax    Calcula e coloca em %eax o valor a devolver

L1:
    movl    -8(%ebp), %ebx
    movl    -4(%ebp), %esi       Recupera o valor dos 2 reg's usados
    movl    %ebp, %esp           Atualiza o valor do stack pointer
    popl    %ebp                 Recupera o valor anterior do frame pointer
    ret
```

Suporte a funções e procedimentos no IA-32 (4)



Análise de exemplos

– revisão do exemplo swap

- análise das fases: arranque/inicialização, corpo, término
- análise dos contextos (IA-32)
- evolução dos contextos na *stack* (IA-32)

– evolução de um exemplo: Fibonacci

- análise de uma compilação do gcc

– aninhamento e recursividade

- evolução dos contextos na *stack*

Exemplo de cadeia de invocações no IA-32 (1)



Estrutura do código

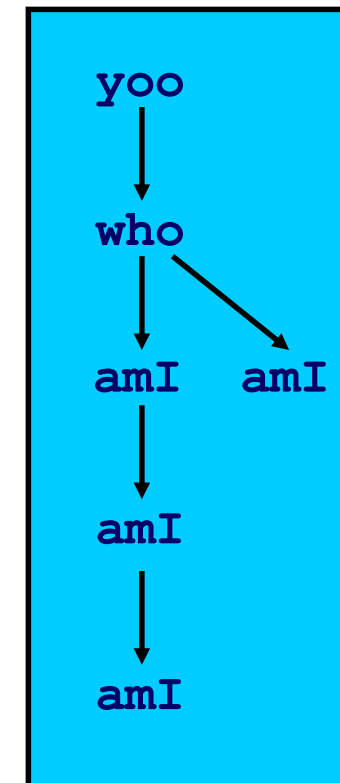
```
yoo (...)  
{  
  .  
  .  
  who () ;  
  .  
  .  
}
```

```
who (...)  
{  
  . . .  
  amI () ;  
  . . .  
  amI () ;  
  . . .  
}
```

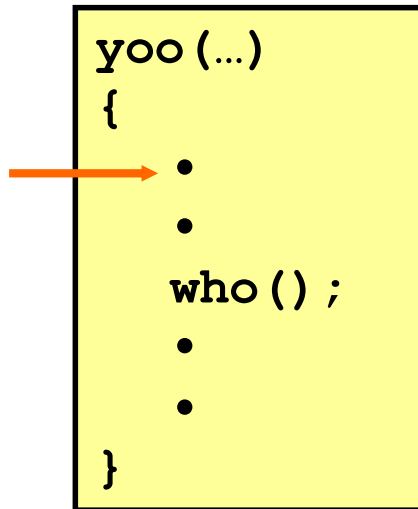
```
amI (...)  
{  
  .  
  .  
  amI () ;  
  .  
  .  
}
```

Função **amI** é recursiva

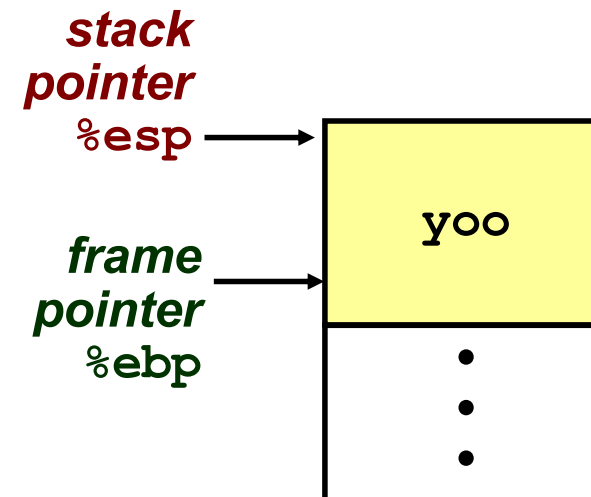
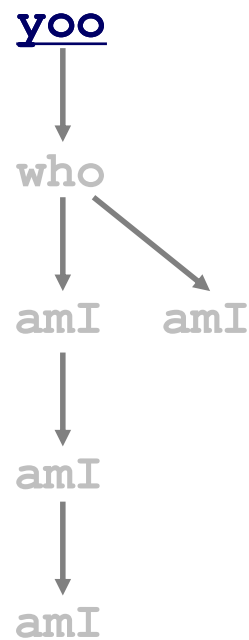
Cadeia de *call*



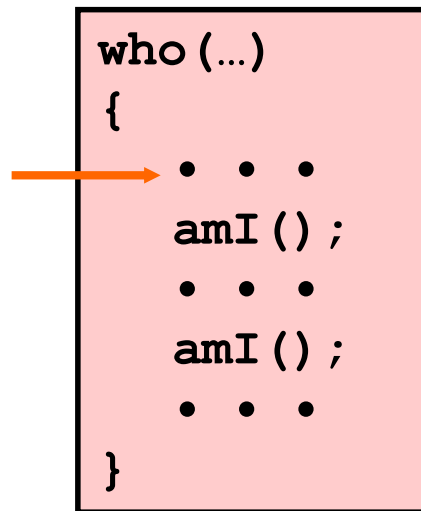
Exemplo de cadeia de invocações no IA-32 (2)



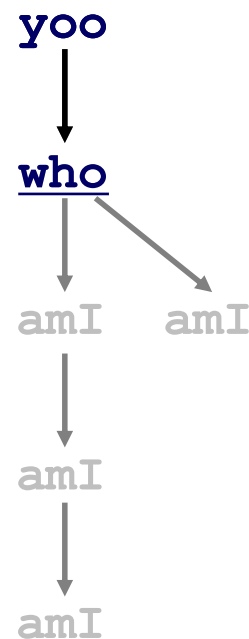
Cadeia de *call*



Exemplo de cadeia de invocações no IA-32 (3)

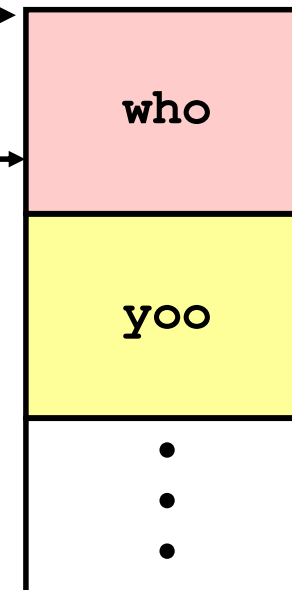


Cadeia de *call*

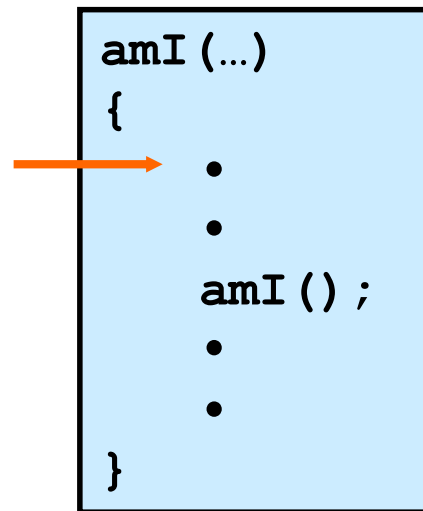


**stack
pointer
%esp**

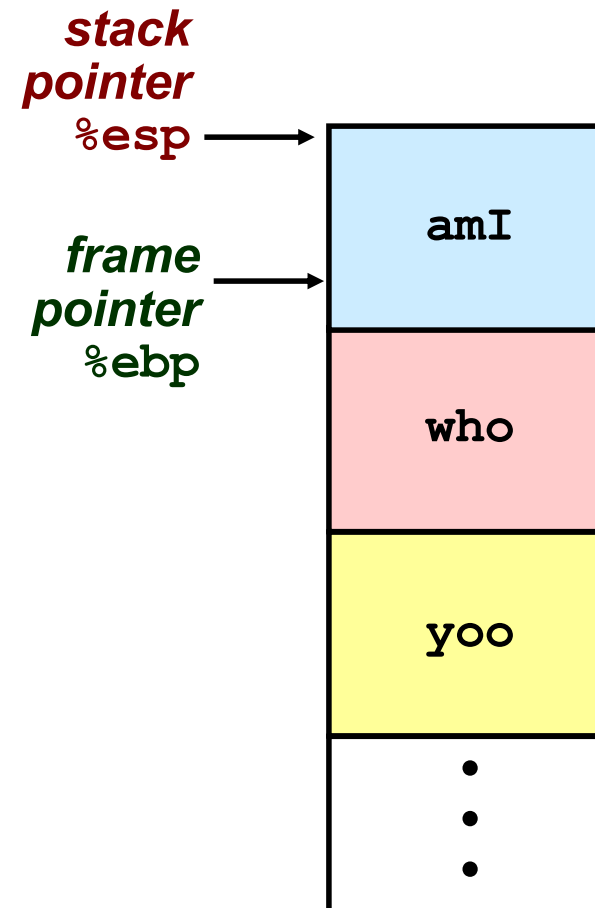
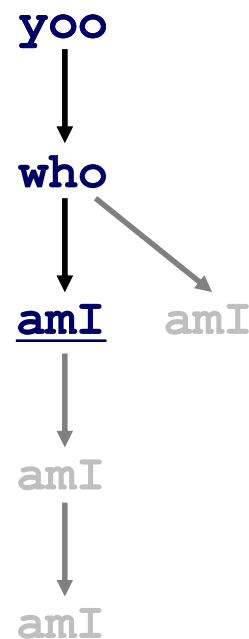
**frame
pointer
%ebp**



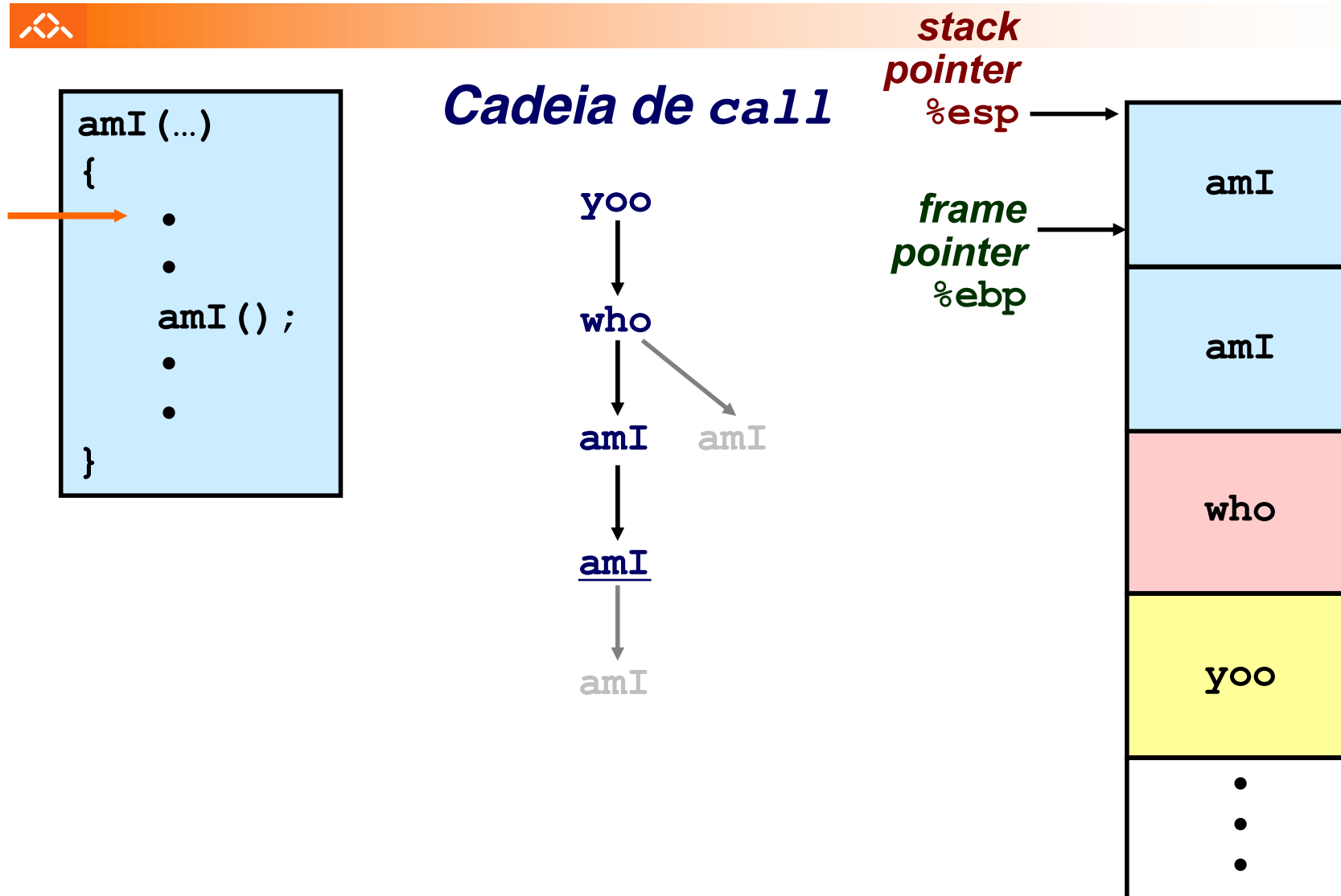
Exemplo de cadeia de invocações no IA-32 (4)



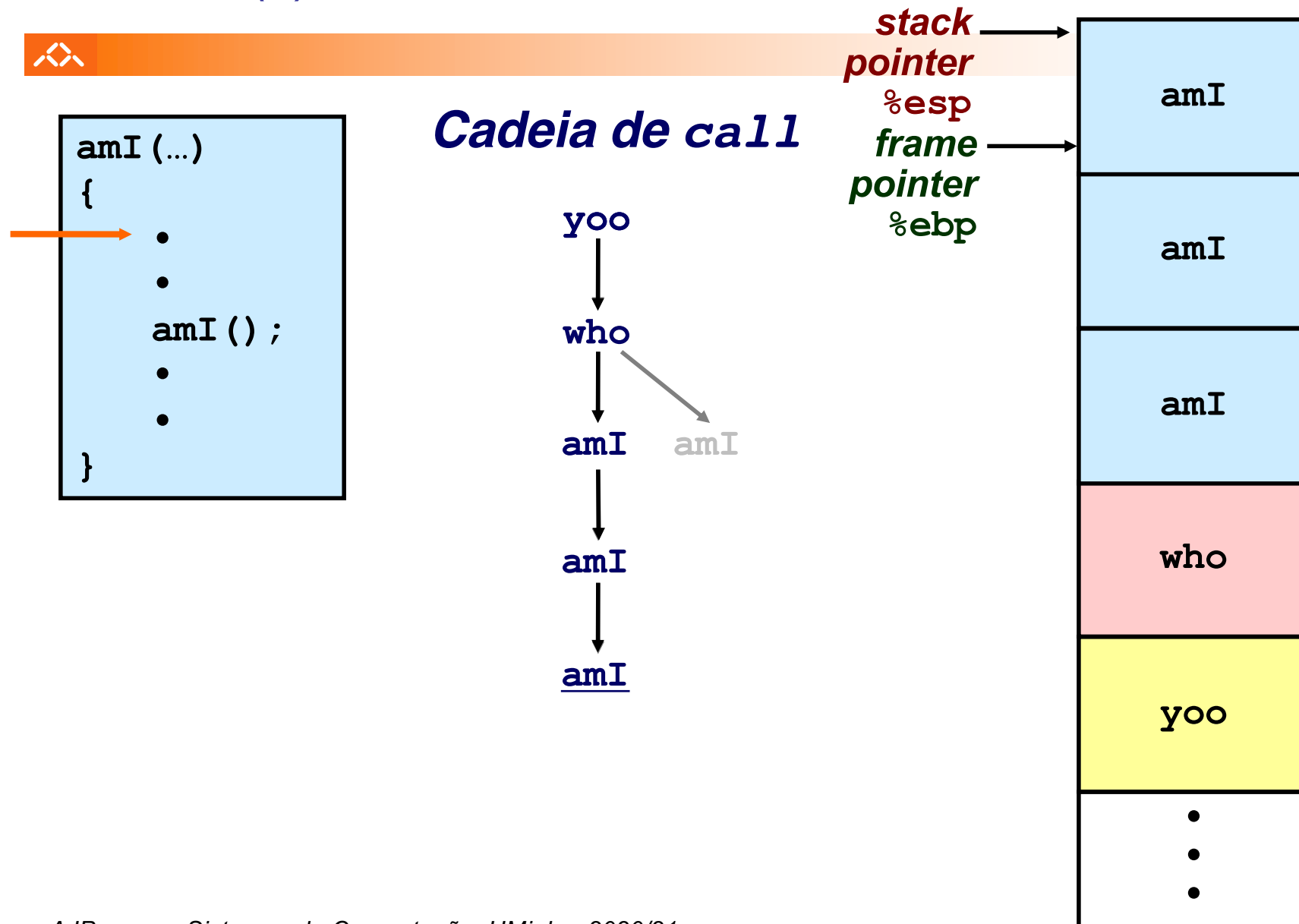
Cadeia de *call*



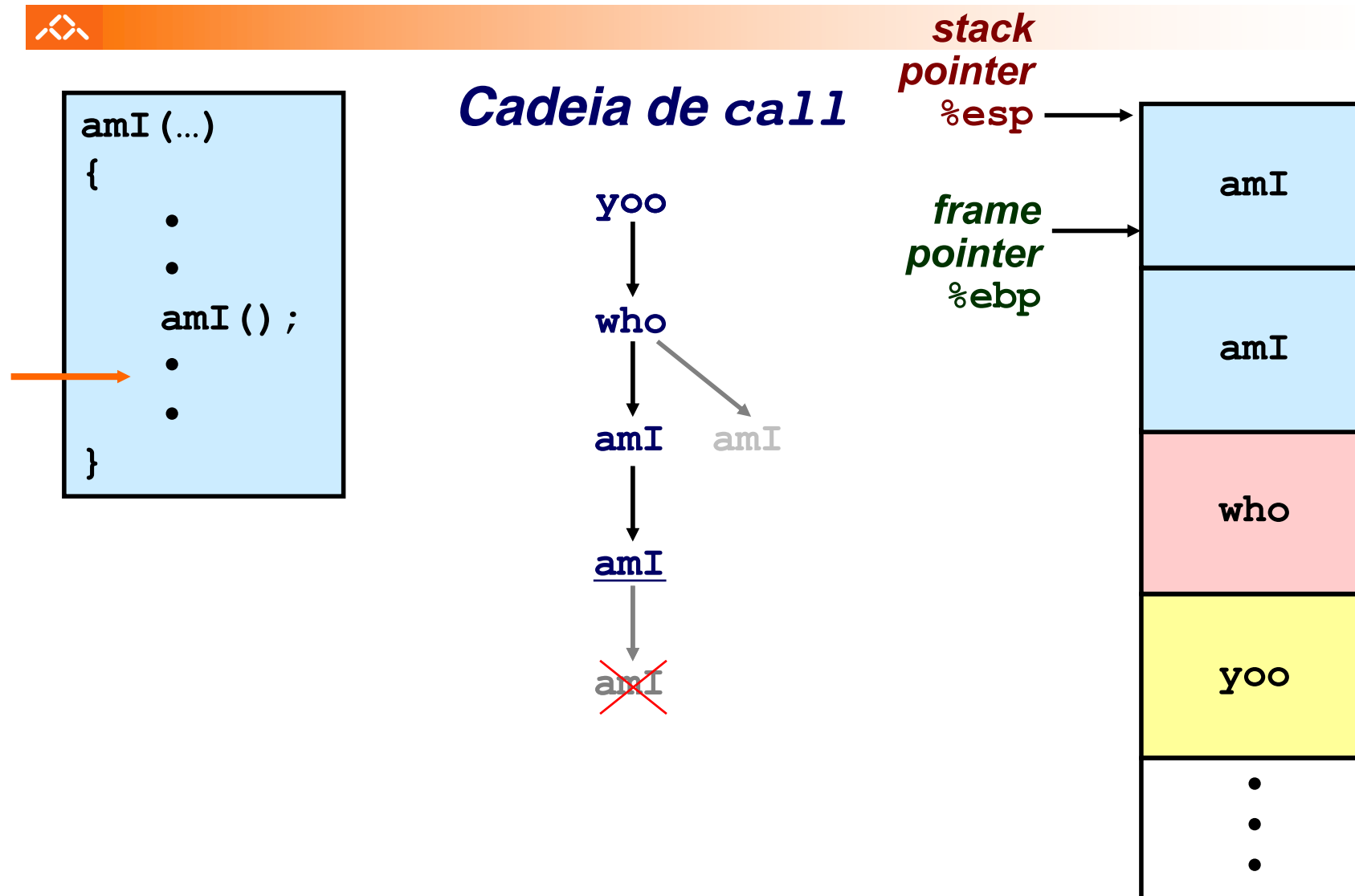
Exemplo de cadeia de invocações no IA-32 (5)



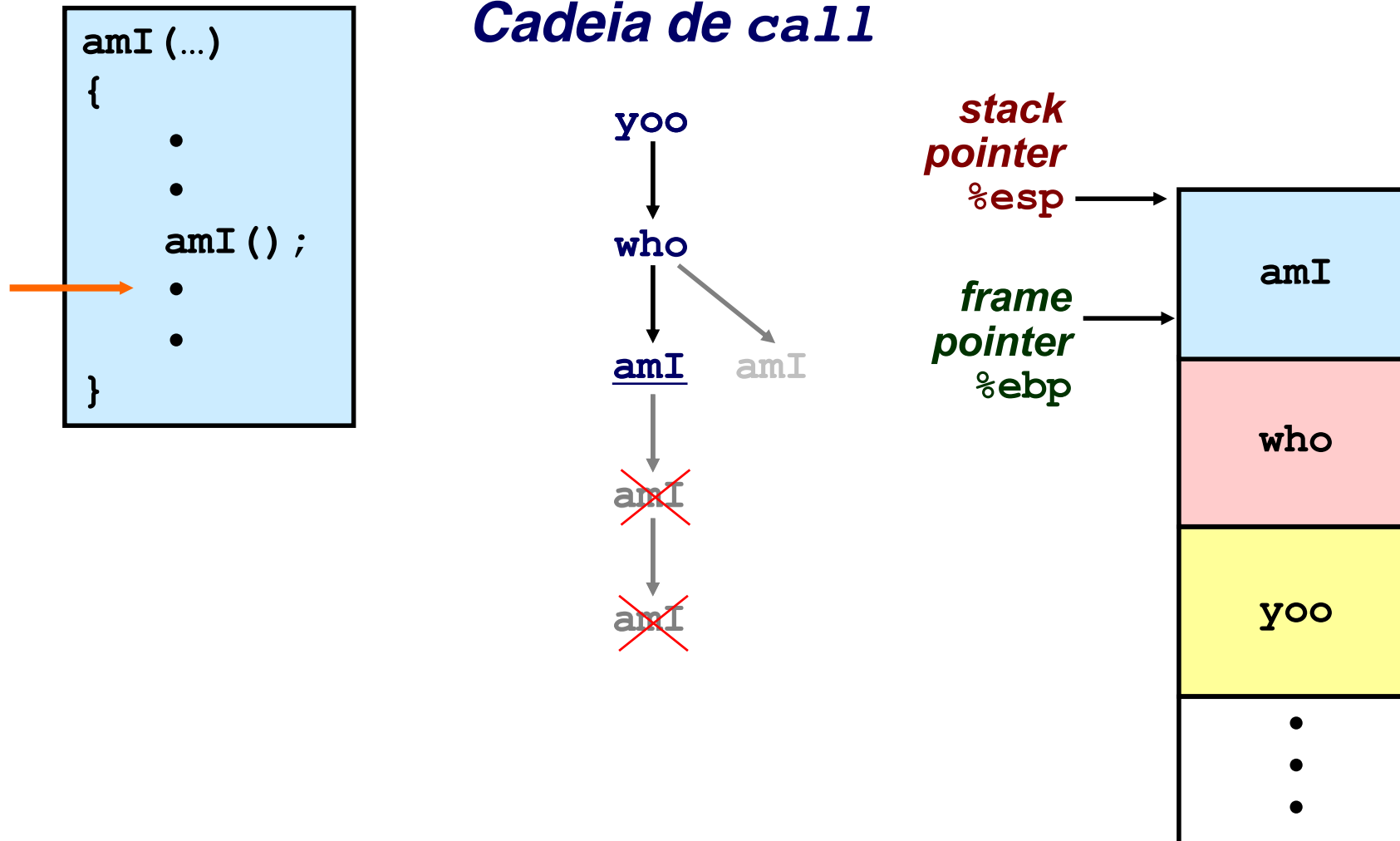
Exemplo de cadeia de invocações no IA-32 (6)



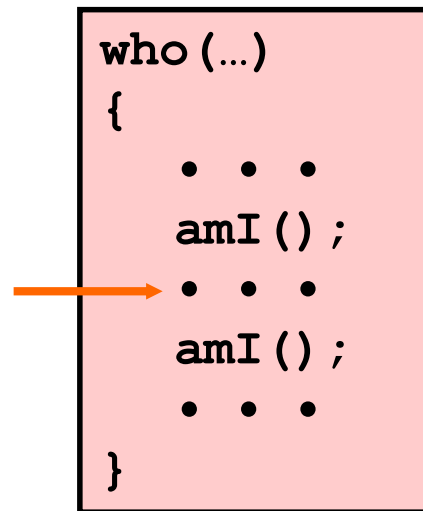
Exemplo de cadeia de invocações no IA-32 (7)



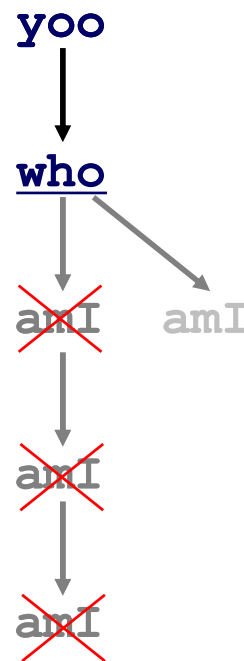
Exemplo de cadeia de invocações no IA-32 (8)



Exemplo de cadeia de invocações no IA-32 (9)

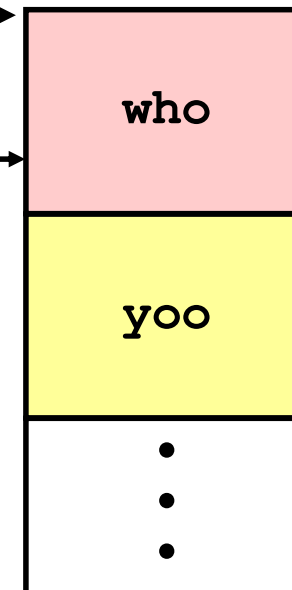


Cadeia de *call*

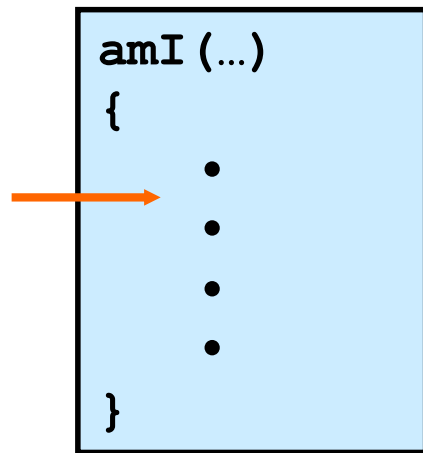


**stack
pointer
%esp**

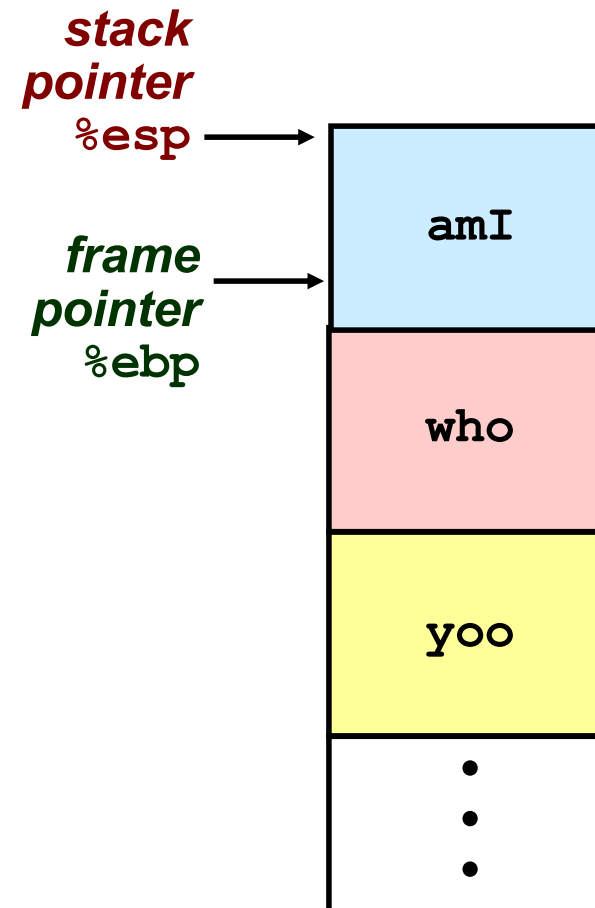
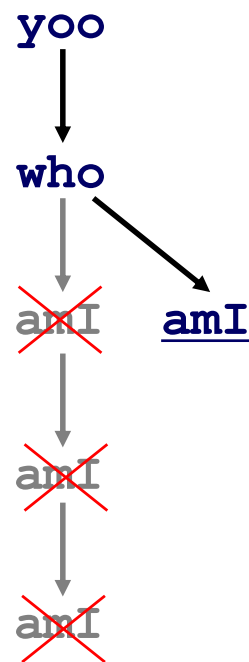
**frame
pointer
%ebp**



Exemplo de cadeia de invocações no IA-32 (10)



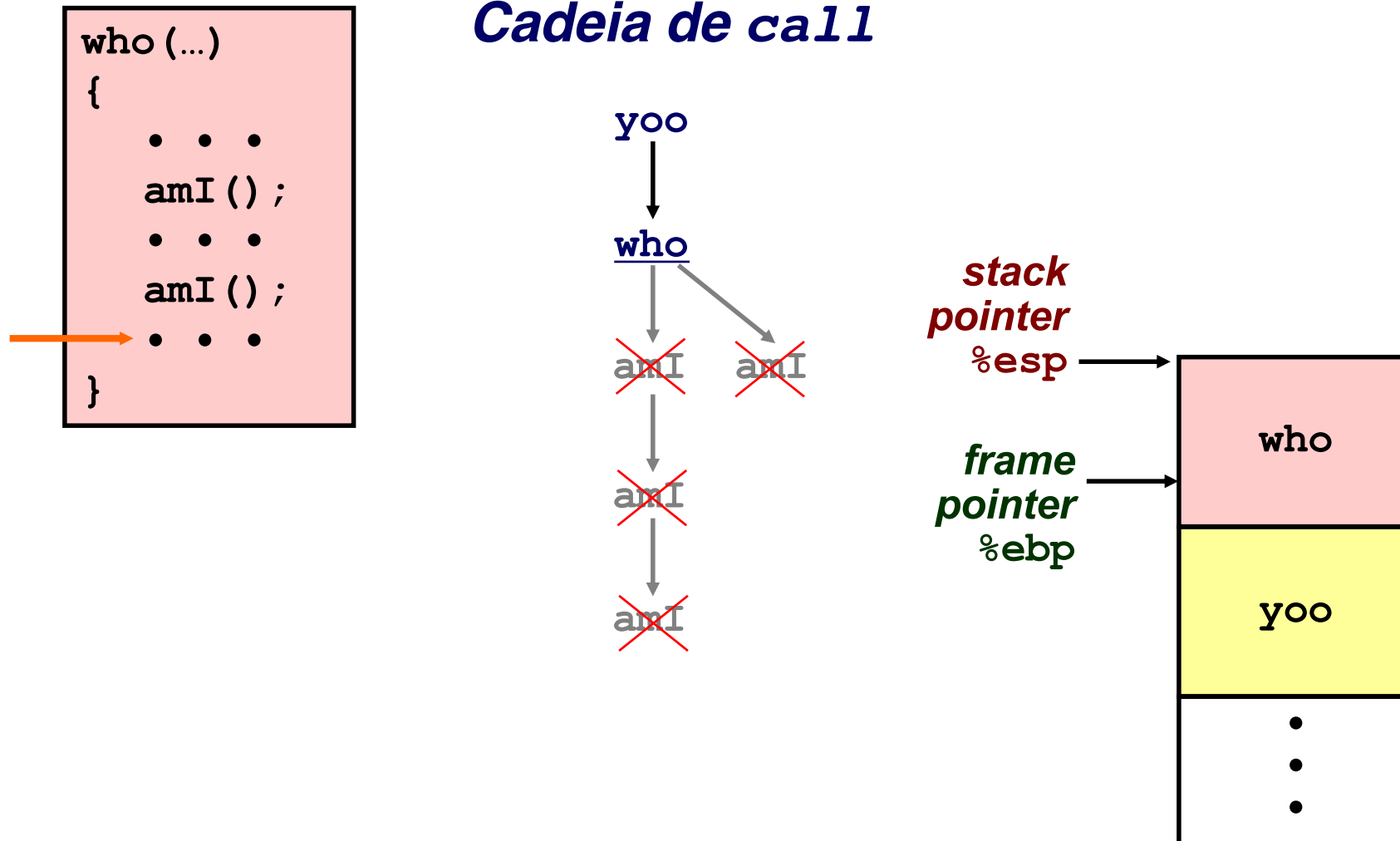
Cadeia de *call*



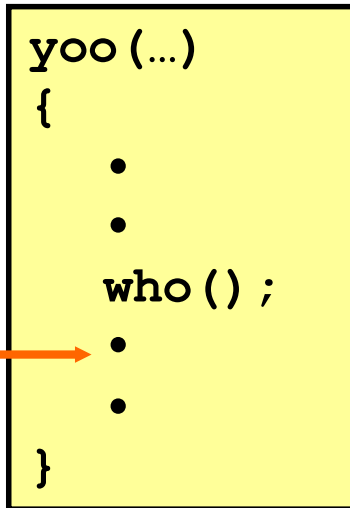
Exemplo de cadeia de invocações no IA-32 (11)



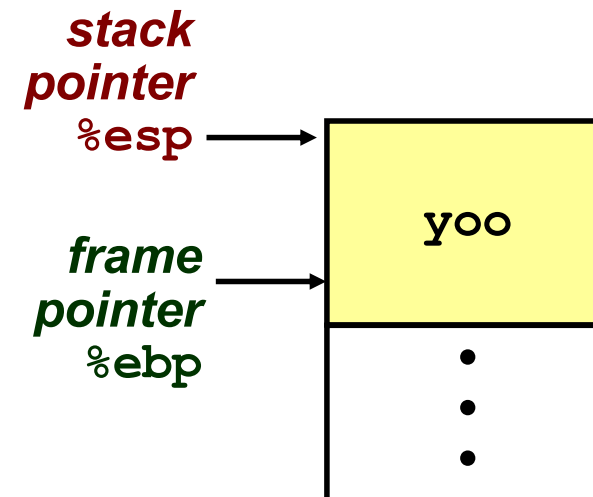
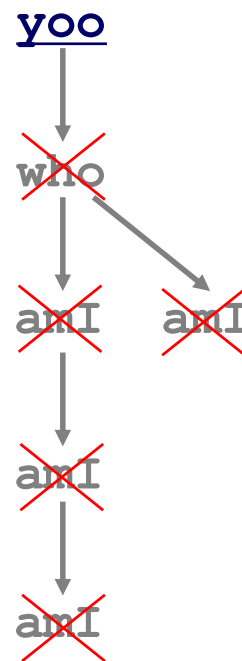
Cadeia de *call*



Exemplo de cadeia de invocações no IA-32 (12)



Cadeia de *call*



Análise duma Instruction Set Architecture (5)



Estrutura do tema ISA do IA-32

1. Desenvolvimento de programas no IA-32 em Linux
2. Acesso a operandos e operações
3. Suporte a estruturas de controlo
4. Suporte à invocação/regresso de funções
5. **Análise comparativa: IA-32 vs. x86-64 e RISC (MIPS e ARM)**
6. Acesso e manipulação de dados estruturados

Relembrando: IA-32 versus Intel 64



Principal diferença na organização interna:

– organização dos registos

- IA-32: poucos registos genéricos (**só 6**) => variáveis locais em reg e argumentos na *stack*
- Intel 64: 16 registos genéricos => mais registos para variáveis locais & para passagem e uso de argumentos (**8 + 6**)

– consequências:

- menor utilização da *stack* na arquitetura Intel 64
- Intel 64 potencialmente mais eficiente

Análise de um exemplo (swap) ...

x86-64: 64-bit extension to IA-32

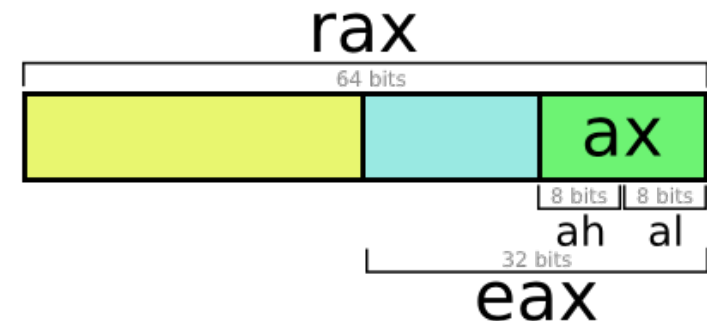
Intel 64: Intel implementation of x86-64



x86-64 Integer Registers

%rax	%eax	%r8	%r8d
%rbx	%ebx	%r9	%r9d
%rcx	%ecx	%r10	%r10d
%rdx	%edx	%r11	%r11d
%rsi	%esi	%r12	%r12d
%rdi	%edi	%r13	%r13d
%rsp	%esp	%r14	%r14d
%rbp	%ebp	%r15	%r15d

- Twice the number of registers
- Accessible as 8, 16, 32, 64 bits



x86-64 Integer Registers: Usage Conventions

%rax	Return value	%r8	Argument #5
%rbx	Callee saved	%r9	Argument #6
%rcx	Argument #4	%r10	Caller saved
%rdx	Argument #3	%r11	Caller Saved
%rsi	Argument #2	%r12	Callee saved
%rdi	Argument #1	%r13	Callee saved
%rsp	Stack pointer	%r14	Callee saved
%rbp	Callee saved	%r15	Callee saved

University of Washington

Relembrando: IA-32 versus Intel 64



Principal diferença na organização interna:

- organização dos registos
 - IA-32: poucos registos genéricos (só 6) => variáveis locais em reg e argumentos na *stack*
 - Intel 64: 16 registos genéricos => mais registos para variáveis locais & para passagem e uso de argumentos (8 + 6)
- consequências:
 - menor utilização da *stack* na arquitetura Intel 64
 - Intel 64 potencialmente mais eficiente

Análise de um exemplo (swap) ...

Revisão da codificação de swap e call_swap no IA-32



```
void swap(int *xp, int *yp)
{
    int t0 = *xp;
    int t1 = *yp;
    *xp = t1;
    *yp = t0;
}
```

```
_swap:
    pushl    %ebp
    movl     %esp, %ebp
    pushl    %ebx

    movl     12(%ebp), %ecx
    movl     8(%ebp), %edx
    movl     (%ecx), %eax
    movl     (%edx), %ebx
    movl     %eax, (%edx)
    movl     %ebx, (%ecx)

    movl     -4(%ebp), %ebx
    movl     %ebp, %esp
    popl     %ebp
    ret
```

```
void call_swap()
{
    int zip1 = 15213;
    int zip2 = 91125;
    swap(&zip1, &zip2);
}
```

```
_call_swap:
    pushl    %ebp
    movl     %esp, %ebp
    subl     $24, %esp

    movl     $15213, -4(%ebp)
    movl     $91125, -8(%ebp)
    leal     -4(%ebp), %eax
    movl     %eax, (%esp)
    leal     -8(%ebp), %eax
    movl     %eax, 4(%esp)
    call     _swap

    movl     %ebp, %esp
    popl     %ebp
    ret
```

Funções em assembly: IA-32 versus Intel 64

	IA-32
_swap:	
pushl %ebp	
movl %esp, %ebp	
pushl %ebx	
movl 8(%ebp), %edx	
movl 12(%ebp), %ecx	
movl (%edx), %ebx	
movl (%ecx), %eax	
movl %eax, (%edx)	
movl %ebx, (%ecx)	
popl %ebx	
popl %ebp	
ret	
_call_swap:	
pushl %ebp	
movl %esp, %ebp	
subl \$24, %esp	
movl \$15213, -4(%ebp)	
movl \$91125, -8(%ebp)	
leal -4(%ebp), %eax	
movl %eax, (%esp)	
leal -8(%ebp), %eax	
movl %eax, 4(%esp)	
call _swap	
movl %ebp, %esp	
popl %ebp	
ret	
Total:	63 bytes

	Intel 64
swap:	
pushq %rbp	
movq %rsp, %rbp	
movl (%rdi), %eax	
movl (%rsi), %ecx	
movl %ecx, (%rdi)	
movl %eax, (%rsi)	
popq %rbp	
retq	
call_swap:	
pushq %rbp	
movq %rsp, %rbp	
subq \$16, %rsp	
movl \$15213, -4(%rbp)	
movl \$91125, -8(%rbp)	
leaq -4(%rbp), %rdi	
leaq -8(%rbp), %rsi	
callq _swap	
addq \$16, %rsp	
popq %rbp	
retq	
Total:	54 bytes

Total de acessos à stack: 15 no IA-32, 9 no Intel 64 !

CISC versus RISC

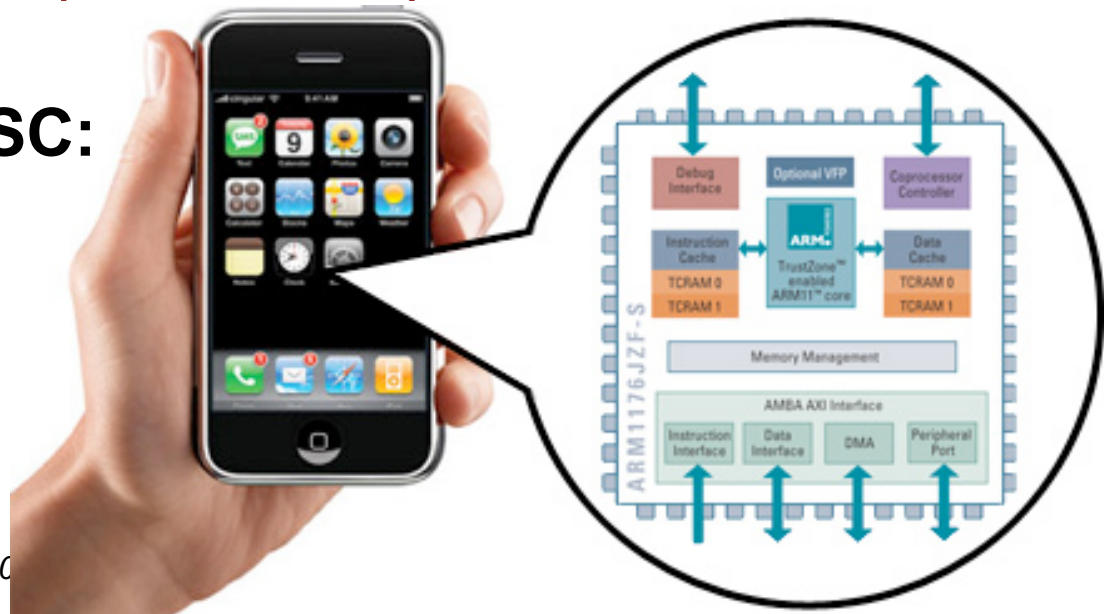


Caracterização das arquiteturas RISC

- conjunto reduzido e simples de instruções
- operandos sempre em registos
- formatos simples de instruções
- modos simples de endereçamento à memória
- uma operação elementar por ciclo máquina

Ex de uma arquitetura RISC:

ARM



Análise do nível ISA: o modelo RISC versus IA-32 (1)



RISC versus IA-32 :

- RISC: conjunto reduzido e simples de instruções
 - pouco mais que o *subset* do IA-32 já apresentado...
 - instruções simples, mas muito eficientes em *pipeline*
- operações aritméticas e lógicas:
 - 3-operandos (RISC) *versus* 2-operandos (IA-32)
 - RISC: operandos sempre em registos,
16/32 registos genéricos visíveis ao programador,
sendo normalmente
 - 1 reg apenas de leitura, com o valor 0 (em 32 registos)
 - 1 reg usado para guardar o endereço de regresso da função
 - 1 reg usado como *stack pointer* (convenção do s/w)

— ■ ■ ■

Análise do nível ISA: o modelo RISC versus IA-32 (2)



RISC versus IA-32 (cont.):

- RISC: modos simples de endereçamento à memória
 - apenas 1 modo de especificar o endereço:
 $\text{Mem} [C^{\text{te}} + (\text{Reg}_b)] \quad \text{ou} \quad \text{Mem} [(\text{Reg}_b) + (\text{Reg}_i)]$
 - ou poucos modos de especificar o endereço:
 $\text{Mem} [C^{\text{te}} + (\text{Reg}_b)] \quad \text{e/ou}$
 $\text{Mem} [(\text{Reg}_b) + (\text{Reg}_i)] \quad \text{e/ou}$
 $\text{Mem} [C^{\text{te}} + (\text{Reg}_b) + (\text{Reg}_i)]$
- RISC: uma operação elementar em cada instrução
 - por ex. `push/pop` (IA-32)
substituído pelo par de operações elementares
`sub&store/load&add` (RISC)

— . . .

Análise do nível ISA: o modelo RISC versus IA-32 (3)



RISC versus IA-32 (cont.):

- RISC: formatos simples de instruções
 - comprimento fixo e poucas variações
 - ex.: MIPS

Name	Fields						Comments
Field size	6 bits	5 bits	5 bits	5 bits	5 bits	6 bits	All MIPS instructions are 32 bits long
R-format	op	rs	rt	rd	shamt	funct	Arithmetic instruction format
I-format	op	rs	rt	address/immediate			Transfer, branch, imm. format
J-format	op	target address					Jump instruction format

- ex.: ARM

Name	Format	Example								Comments
Field size		4 bits	2 bits	1 bit	4 bits	1 bit	4 bits	4 bits	12 bits	All ARM instructions are 32 bits long
DP format	DP	Cond	F	I	Opcode	S	Rn	Rd	Operand2	Arithmetic instruction format
DT format	DT	Cond	F	Opcode			Rn	Rd	Offset12	Data transfer format
Field size		4 bits	2 bits	2 bits	24 bits					
BR format	BR	Cond	F	Opcode	signed_immed_24					B and BL instructions

Simples comparação entre ARM e MIPS

- ARM: the most popular embedded core
- Similar basic set of instructions to MIPS

	ARM	MIPS
Date announced	1985	1985
Instruction size	32 bits	32 bits
Address space	32-bit flat	32-bit flat
Data alignment	Aligned	Aligned
Data addressing modes	9	3
Registers	15 × 32-bit	31 × 32-bit
Input/output	Memory mapped	Memory mapped

IA-32 versus RISC (1)



Principal diferença na organização interna:

– organização dos registos

- IA-32: poucos registos genéricos => variáveis e argumentos normalmente na *stack*
- RISC: 16/32 registos genéricos => mais registos para variáveis locais, & registos para passagem de argumentos & registo para endereço de regresso

– consequências:

- menor utilização da *stack* nas arquiteturas RISC
- RISC potencialmente mais eficiente

Análise de um exemplo (*swap*) ...

IA-32 versus RISC (2)



Principal diferença na organização interna:

- organização dos registos
 - IA-32: poucos registos genéricos => variáveis e argumentos normalmente na *stack*
 - RISC: 16/32 registos genéricos => mais registos para variáveis locais, & registos para passagem de argumentos & registo para endereço de regresso
- consequências:
 - menor utilização da *stack* nas arquiteturas RISC
 - RISC potencialmente mais eficiente

Análise de um exemplo (swap) ...

Revisão da codificação de swap e call_swap no IA-32



```
void swap(int *xp, int *yp)
{
    int t0 = *xp;
    int t1 = *yp;
    *xp = t1;
    *yp = t0;
}
```

```
_swap:
    pushl    %ebp
    movl     %esp, %ebp
    pushl    %ebx

    movl     12(%ebp), %ecx
    movl     8(%ebp), %edx
    movl     (%ecx), %eax
    movl     (%edx), %ebx
    movl     %eax, (%edx)
    movl     %ebx, (%ecx)

    movl     -4(%ebp), %ebx
    movl     %ebp, %esp
    popl     %ebp
    ret
```

```
void call_swap()
{
    int zip1 = 15213;
    int zip2 = 91125;
    swap(&zip1, &zip2);
}
```

```
_call_swap:
    pushl    %ebp
    movl     %esp, %ebp
    subl     $24, %esp

    movl     $15213, -4(%ebp)
    movl     $91125, -8(%ebp)
    leal     -4(%ebp), %eax
    movl     %eax, (%esp)
    leal     -8(%ebp), %eax
    movl     %eax, 4(%esp)
    call     _swap

    movl     %ebp, %esp
    popl     %ebp
    ret
```


Convenção na utilização dos registos MIPS



MIPS

Name	Register	Usage
\$zero	\$0	Always 0 (forced by hardware)
\$at	\$1	Reserved for assembler use
\$v0 – \$v1	\$2 – \$3	Result values of a function
\$a0 – \$a3	\$4 – \$7	Arguments of a function
\$t0 – \$t7	\$8 – \$15	Temporary Values
\$s0 – \$s7	\$16 – \$23	Saved registers (preserved across call)
\$t8 – \$t9	\$24 – \$25	More temporaries
\$k0 – \$k1	\$26 – \$27	Reserved for OS kernel
\$gp	\$28	Global pointer (points to global data)
\$sp	\$29	Stack pointer (points to top of stack)
\$fp	\$30	Frame pointer (points to stack frame)
\$ra	\$31	Return address (used by jal for function call)

Funções em assembly: IA-32 versus MIPS (RISC) (1)

 <pre> _swap: pushl %ebp movl %esp, %ebp pushl %ebx movl 8(%ebp), %edx movl 12(%ebp), %ecx movl (%edx), %ebx movl (%ecx), %eax movl %eax, (%edx) movl %ebx, (%ecx) popl %ebx popl %ebp ret _call_swap: pushl %ebp movl %esp, %ebp subl \$24, %esp movl \$15213, -4(%ebp) movl \$91125, -8(%ebp) leal -4(%ebp), %eax movl %eax, (%esp) leal -8(%ebp), %eax movl %eax, 4(%esp) call _swap movl %ebp, %esp popl %ebp ret </pre> IA-32 Total: 63 bytes	<pre> swap: lw \$v1, 0(\$a0) lw \$v0, 0(\$a1) sw \$v0, 0(\$a0) sw \$v1, 0(\$a1) j \$ra call_swap: subu \$sp, \$sp, 32 sw \$ra, 24(\$sp) li \$v0, 15213 sw \$v0, 16(\$sp) li \$v0, 0x10000 ori \$v0, \$v0, 0x63f5 sw \$v0, 20(\$sp) addu \$a0, \$sp, 16 # &zip1= sp+16 addu \$a1, \$sp, 20 # &zip2= sp+20 jal swap lw \$ra, 24(\$sp) addu \$sp, \$sp, 32 j \$ra </pre> MIPS Total: 72 bytes

Funções em assembly: IA-32 versus MIPS (RISC) (2)



call_swap

1. Invocar swap

- salvaguardar registros
- passagem de argumentos
- chamar rotina e guardar endereço de regresso

```
leal    -4(%ebp), %eax
pushl   %eax
leal    -8(%ebp), %eax
pushl   %eax
call    swap
```

Não há reg para salvag.

Calcula &zip2

Push &zip2

Calcula &zip1

Push &zip1

Invoca swap

IA-32

**Acessos
à stack**

MIPS

```
sw      $ra, 24($sp)
```

```
addu    $a0, $sp, 16
```

```
addu    $a1, $sp, 20
```

```
jal     swap
```

Salvag. reg c/ endereço regresso

Calcula & coloca &zip1 no reg arg 0

Calcula & coloca &zip2 no reg arg 1

Invoca swap

Funções em assembly: IA-32 versus MIPS (RISC) (3)



swap

1. Inicializar swap

- atualizar *frame pointer*
- salvar registos
- reservar espaço p/ locais

swap:

pushl %ebp

movl %esp, %ebp

pushl %ebx

Salvag. antigo %ebp

%ebp novo frame pointer

Salvag. %ebx

Não é preciso espaço p/ locais

IA-32

Acessos
à stack

MIPS

Frame pointer p/ atualizar: NÃO

Registos p/ salvar: NÃO

Espaço p/ locais: NÃO

Funções em assembly: IA-32 versus MIPS (RISC) (4)



swap

2. Corpo de swap ...

```
movl 12(%ebp), %ecx
movl 8(%ebp), %edx
movl (%ecx), %eax
movl (%edx), %ebx
movl %eax, (%edx)
movl %ebx, (%ecx)
```

*Coloca **yp** em reg*
*Coloca **xp** em reg*
*Coloca **y** em reg*
*Coloca **x** em reg*
*Armazena **y** em ***xp***
*Armazena **x** em ***yp***

IA-32

Acessos
à memória
(todas...)

MIPS

```
lw $v1, 0($a0)
lw $v0, 0($a1)
sw $v0, 0($a0)
sw $v1, 0($a1)
```

*Coloca **x** em reg*
*Coloca **y** em reg*
*Armazena **y** em ***xp***
*Armazena **x** em ***yp***

Funções em assembly: IA-32 versus MIPS (RISC) (5)



swap

3. Término de swap ...

- libertar espaço de var locais
- recuperar registos
- recuperar antigo *frame pointer*
- regressar a *call_swap*

```
popl %ebx  
movl %ebp, %esp  
popl %ebp  
ret
```

Não há espaço a libertar

Recupera %ebx

Recupera %esp

Recupera %ebp

Regressa à função chamadora

IA-32

Acessos
à stack

MIPS

```
j $ra
```

Espaço a libertar de var locais: NÃO

Recuperação de registos: NÃO

Recuperação do *frame ptr*: NÃO

Regressa à função chamadora

Funções em assembly: IA-32 versus MIPS (RISC) (6)



call_swap

2. Terminar invocação de swap...

- libertar espaço de argumentos na *stack*...
- recuperar registos

```
addl $8, (%esp)
```

Atualiza stack pointer
Não há reg's a recuperar

IA-32

**Acessos
à stack**

MIPS

```
lw $ra, 24($sp)
```

Espaço a libertar na stack: NÃO
Recupera reg c/ ender regresso

Total de acessos à memória/stack (incl. inicialização de var's em mem):

14(+2) no IA-32, 6(+2) no MIPS !

Convenção na utilização dos registos ARM



ARM

Name	Register number	Usage	Preserved on call?
a1 - a2	0-1	Argument / return result / scratch register	no
a3 - a4	2-3	Argument / scratch register	no
v1 - v8	4-11	Variables for local routine	yes
ip	12	Intra-procedure-call scratch register	no
sp	13	Stack pointer	yes
lr	14	Link Register (Return address)	yes
pc	15	Program Counter	n.a.

Funções em assembly: IA-32 versus ARM (RISC)



<pre> _swap: pushl %ebp movl %esp, %ebp pushl %ebx movl 8(%ebp), %edx movl 12(%ebp), %ecx movl (%edx), %ebx movl (%ecx), %eax movl %eax, (%edx) movl %ebx, (%ecx) popl %ebx popl %ebp ret _call_swap: pushl %ebp movl %esp, %ebp subl \$24, %esp movl \$15213, -4(%ebp) movl \$91125, -8(%ebp) leal -4(%ebp), %eax movl %eax, (%esp) leal -8(%ebp), %eax movl %eax, 4(%esp) call _swap movl %ebp, %esp popl %ebp ret </pre>	IA-32	Total: 63 bytes
---	---	--

<pre> _swap: str fp, [sp, #-4]! add fp, sp, #0 ; IA-32: mov sp, fp ldr r3, [r0, #0] ; IA-32: mov 0(r0), r3 ldr r2, [r1, #0] str r2, [r0, #0] ; IA-32: mov r2, 0(r0) str r3, [r1, #0] add sp, fp, #0 pop {fp} ; pop é pseudo-instr bl lr ; branch & link _call_swap: push {fp, lr} ; push é pseudo-instr add fp, sp, #4 sub sp, sp, #8 ldr r3, .L3 str r3, [fp, #-12] ldr r3, .L3+4 str r3, [fp, #-8] sub r0, fp, #12 ; &zip1= fp+12 sub r1, fp, #8 ; &zip2= fp+8 bl _swap sub sp, fp, #4 pop {fp, pc} ; pop {pc} = ret .L3: .word 15213 .word 91125 </pre>	ARM
--	---

GCC (GNU Compiler Collection): linguagens de programação suportadas e ...



Linguagens de programação que GCC suporta

- C com dialetos:
 - ANSI C original (X3.159-1989, ISO standard ISO/IEC 9899:1990) aka **C89** ou **C90**; usar com `'-ansi'`, `'-std=c90'`
 - **C94** ou **C95**, usar com `'-std=iso9899:199409'`
 - **C99**, usar com `'-std=c99'` or `'-std=iso9899:1999'`
 - **C11**, usar com `'-std=c11'`
 - **C17**, usar com `'-std=c17'`
 - por omissão GCC usa **C11** com extensões `'-std=gnu11'`
- C++ (usado com comando `'g++'`) com dialetos:
 - **C++98**, **C++03**, **C++11**, **C++14**, **C++17**, ...
- Fortran (usado com comando `'gfortran'`) com dialetos:
 - **Fortran 95**, **Fortran 2003**, **Fortran 2008**, ...

GCC (GNU Compiler Collection): ... e processadores suportados



Processadores que GCC suporta

- especificado sempre com opção `'-m'`
- opção x86, usar com `'-march=cpu-type'`; algumas escolhas:
 - `'native'` (o sistema local)
 - `'i386'`... `'x86-64'`... `'kn1'`... `'icelake'`... `'znver3'`
- opção MIPS, usar com `'-march=arch'`
 - é possível ainda selecionar um dado processador MIPS
- opção ARM, usar com `'-march=name'`
 - é possível ainda selecionar um dado processador ARM; mais comuns:
 - os da última geração de 32 bits, `'armv7'`
 - da 1ª geração de 64 bits, `'armv8-a'`

Análise do Instruction Set Architecture (6)



Estrutura do tema ISA do IA-32

1. Desenvolvimento de programas no IA-32 em Linux
2. Acesso a operandos e operações
3. Suporte a estruturas de controlo
4. Suporte à invocação/regresso de funções
5. Análise comparativa: IA-32 vs. x86-64 e RISC (MIPS e ARM)
6. Acesso e manipulação de dados estruturados

Dados estruturados em C



Propriedades dos dados estruturados em C

- agregam quantidades escalares do mesmo tipo ou de tipos diferentes
- por norma, alocadas a posições contíguas da memória
- a estrutura definida é referenciada pelo apontador para a 1ª posição de memória

Tipos de dados estruturados mais comuns em C

- **array**: agregado de dados escalares do mesmo tipo
 - *string*: array de caracteres terminado com *null*
 - *arrays de arrays*: arrays multi-dimensionais
- **structure**: agregado de dados de tipos diferentes
 - *structures de structures, structures de arrays, ...*
- **union**: mesmo objecto mas com visibilidade distinta

Arrays: alocação em memória

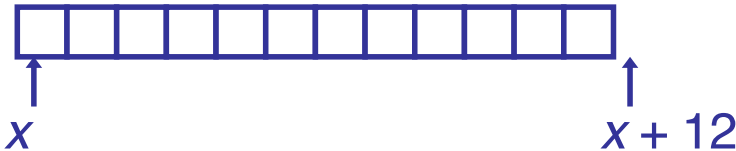


Declaração em C:

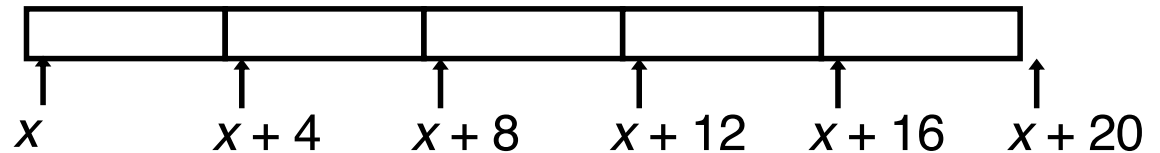
`data_type Array_name[length];`

Aloca em memória uma região com tamanho
 $length * \text{sizeof}(data_type)$ bytes

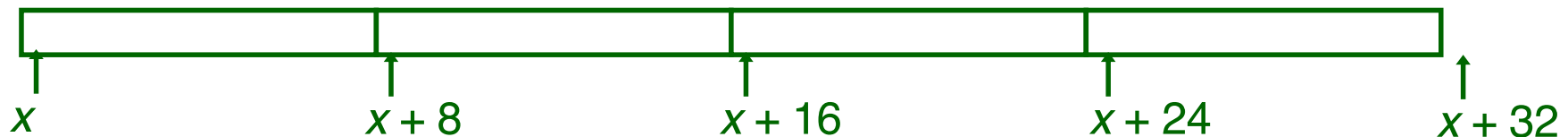
`char string[12];`



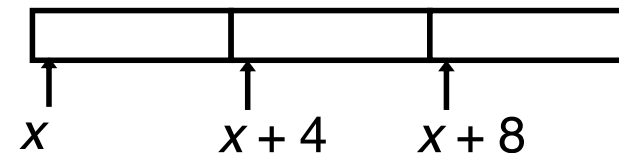
`int val[5];`



`double a[4];`



`char *p[3];`



Arrays:

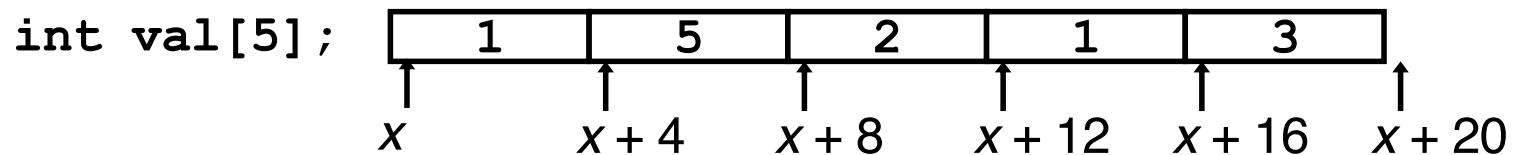
acesso aos elementos



Declaração em C:

```
data_type Array_name [length];
```

O identificador **Array_name** pode ser usado
como apontador para o elemento 0

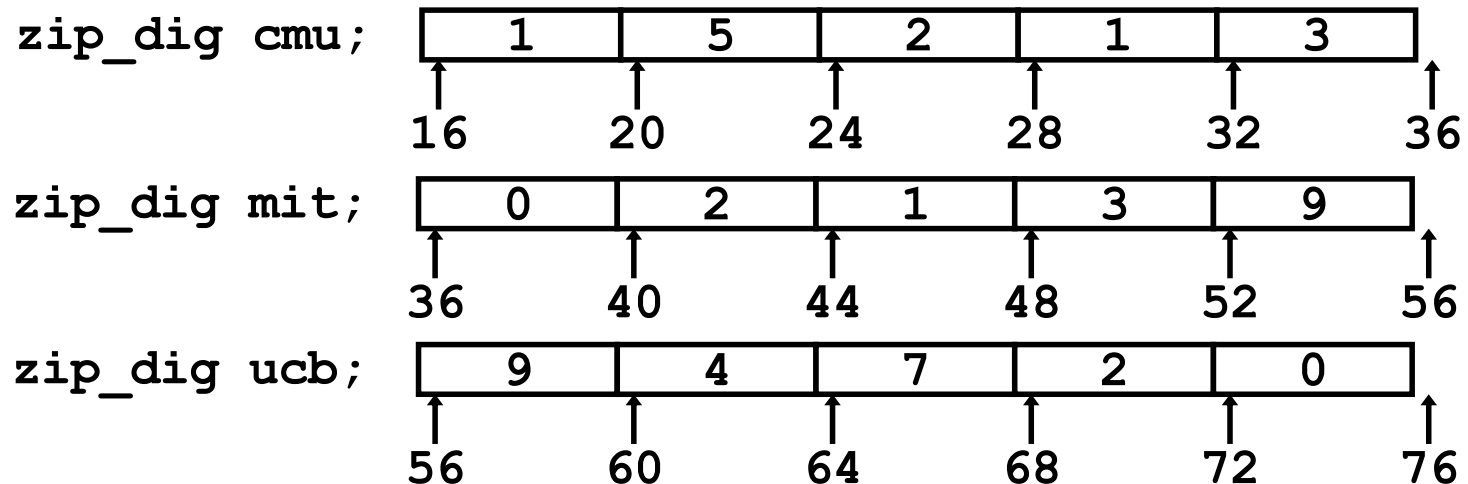


Referência	Tipo	Valor
<code>val[4]</code>	<code>int</code>	3
<code>val</code>	<code>int *</code>	<code>x</code>
<code>val+1</code>	<code>int *</code>	<code>x + 4</code>
<code>&val[2]</code>	<code>int *</code>	<code>x + 8</code>
<code>val[5]</code>	<code>int</code>	??
<code>*(val+1)</code>	<code>int</code>	5
<code>val + i</code>	<code>int *</code>	<code>x + 4 i</code>

Arrays: análise de um exemplo



```
typedef int zip_dig[5];  
  
zip_dig cmu = { 1, 5, 2, 1, 3 };  
zip_dig mit = { 0, 2, 1, 3, 9 };  
zip_dig ucb = { 9, 4, 7, 2, 0 };
```



Notas

- declaração “zip_dig cmu” equivalente a “int cmu[5]”
- os *arrays* deste exemplo ocupam blocos sucessivos de 20 bytes

Arrays no IA-32: exemplo de acesso a um elemento



```
int get_digit(zip_dig z, int dig)
{
    return z[dig];
}
```

Argumentos:

- início do *array* *z* : neste exemplo, o gcc coloca em `%edx`
- índice *dig* do *array* *z* : neste exemplo, o gcc coloca em `%eax`
- a devolver pela função: tipo `int` (4 *bytes*), por convenção, em `%eax`

Localização do elemento ***z[dig]***:

- na memória, em `Mem[(início_array_z) + (índice_dig) * 4]`
- na sintaxe do *assembler* da GNU para IA-32/Linux: em `(%edx, %eax, 4)`

```
movl (%edx,%eax,4),%eax    # %edx <= z
                           # %eax <= dig
                           # devolve z[dig]
```

Arrays no IA-32: apontadores em vez de índices



Análise do código compilado

- Registos

`%ecx` `z`
`%eax` `zi` partilhado com `*z`
`%ebx` `zend`

- Cálculos

- $10 * zi + *z \Rightarrow *z + 2 * (zi + 4 * zi)$
- `z++` incrementa 4

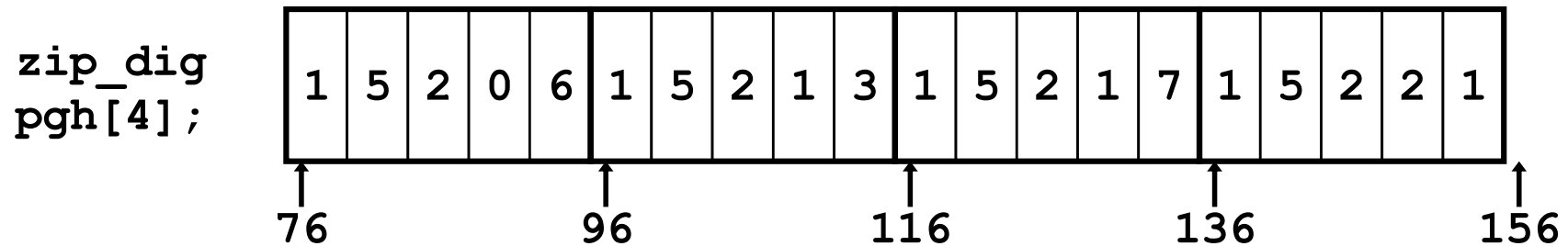
```
int zd2int(zip_dig z)
{
    int zi = 0;
    int *zend = z + 4;
    do {
        zi = 10 * zi + *z;
        z++;
    } while(z <= zend);
    return zi;
}
```

```
                                # %ecx <= z
                                # zi = 0
                                # zend = z+4
                                #loop:
xorl %eax,%eax                  # %edx <= 5*zi
leal 16(%ecx),%ebx              # %eax <= *z
.L59:                           # %ecx <= z++
leal (%eax,%eax,4),%edx         # zi = *z + 2*(5*zi)
movl (%ecx),%eax               # comp z : zend
addl $4,%ecx                   # if <= goto loop
leal (%eax,%edx,2),%eax
cmpl %ebx,%ecx
jle .L59
```

Array de arrays: análise de um exemplo



```
#define PCOUNT 4
zip_dig pgh[PCOUNT] =
    {{1, 5, 2, 0, 6},
     {1, 5, 2, 1, 3 },
     {1, 5, 2, 1, 7 },
     {1, 5, 2, 2, 1 }};
```



- Declaração “zip_dig pgh[4]” equivalente a “int pgh[4][5]”
 - variável pgh é um *array* de 4 elementos
 - alocados em memória em blocos contíguos
 - cada elemento é um *array* de 5 int's
 - alocados em memória em células contínuas
- Ordenação dos elementos em memória (típico em C): “*Row-Major*”

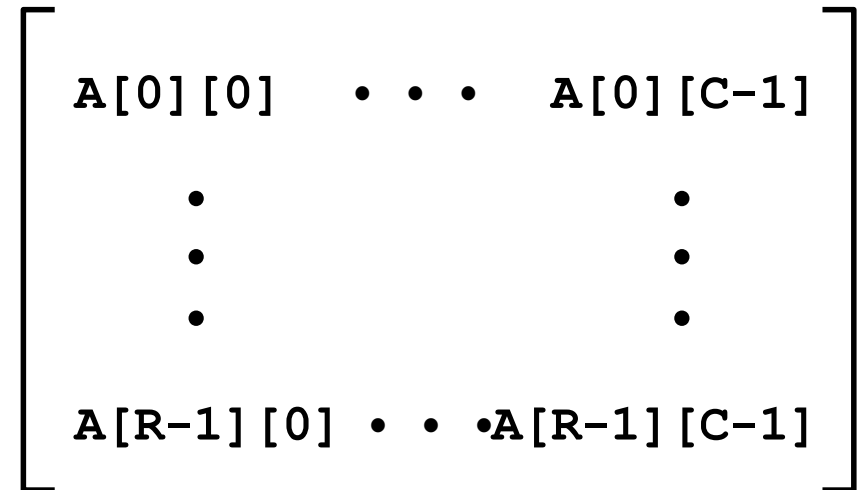
Array de arrays: alocação em memória



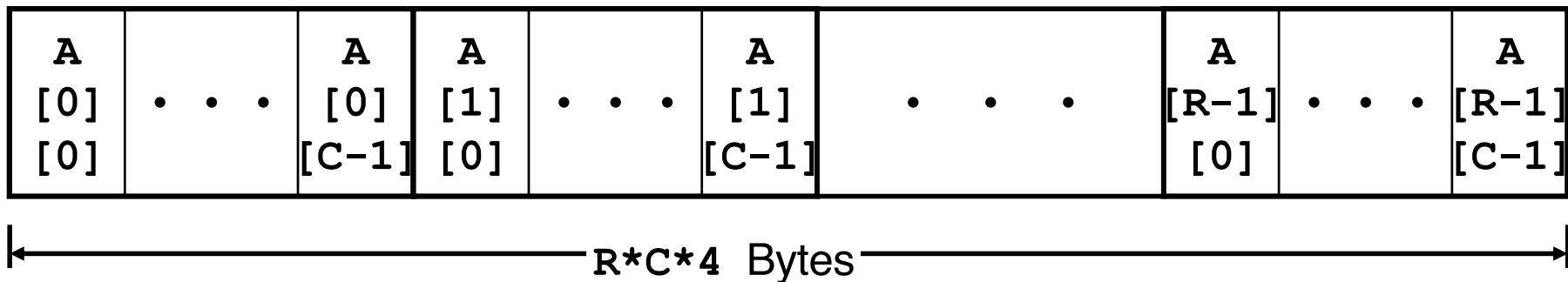
Declaração em C:

`data_type Array_name [R] [C];`

- Alocação em memória de uma região com
 $R * C * \text{sizeof}(\text{data_type})$ bytes
- Ordenação
Row-Major



`int A[R][C];`



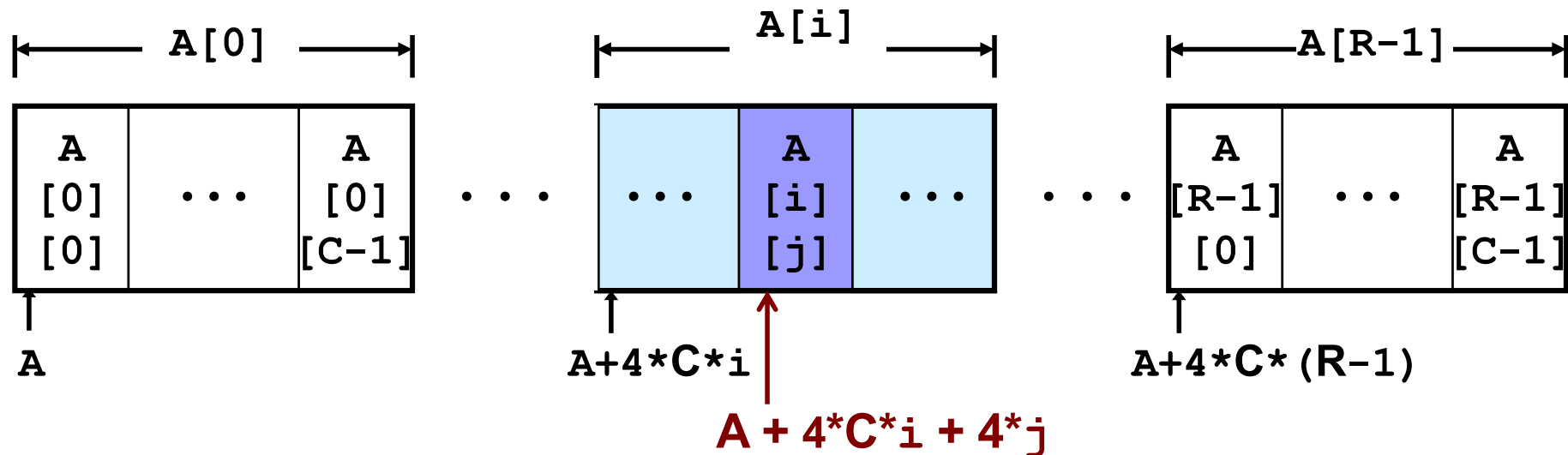
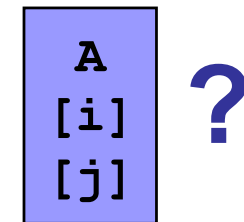
Array de arrays: acesso a um elemento



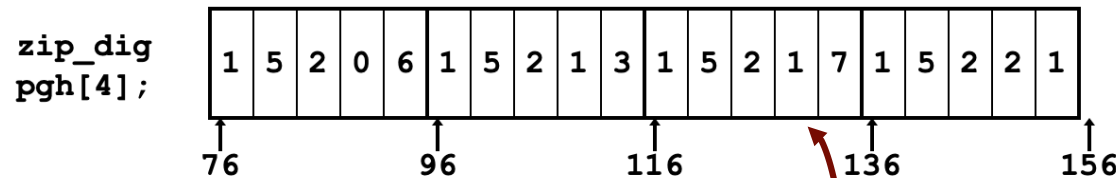
Elementos de um array $R \times C$

- $A[i][j]$ é um elemento do tipo T (*data_type*) com dimensão $K = \text{sizeof}(T)$
- sua localização:
 $A + K * C * i + K * j$

```
int A[R][C];
```



Array de arrays no IA-32: código para acesso a um element (1)



- Declaração "zip_dig pgh[4]" equivalente a "int pgh[4][5]"

Ex: dígito 3 do índice 2 de pgh

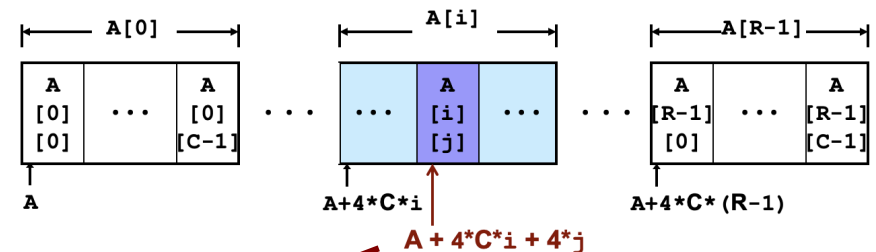
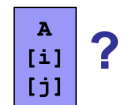
Localização:

$$76 + 20 * 2 + 4 * 3 = 128$$

Elementos de um array R*C

- $A[i][j]$ é um elemento do tipo $T(data_type)$ com dimensão $K = \text{sizeof}(T)$
- sua localização:
 $A + K * C * i + K * j$

int A[R][C];



- Localização em memória de
pgh[index][dig]:
 $\text{pgh} + 20 * \text{index} + 4 * \text{dig}$

```
int get_pgh_digit
(int index, int dig)
{
    return pgh[index][dig];
}
```

Array de arrays no IA-32: código para acesso a um element (2)



- **Localização em memória de**

`pgh[index][dig]:`

`pgh + 20*index + 4*dig`

- **Código em *assembly*:**

- cálculo do endereço

`pgh + 4*(index+4*index) + 4*dig`

- acesso ao elemento: com `movl`

```
int get_pgh_digit
(int index, int dig)
{
    return pgh[index][dig];
}
```

```
                                # %ecx = dig
                                # %eax = index
leal 0(,%ecx,4),%edx           # %edx = 4*dig
leal (%eax,%eax,4),%eax        # %eax = 5*index
movl pgh(%edx,%eax,4),%eax     # devolve Mem(pgh+4*5*index+4*dig)
```

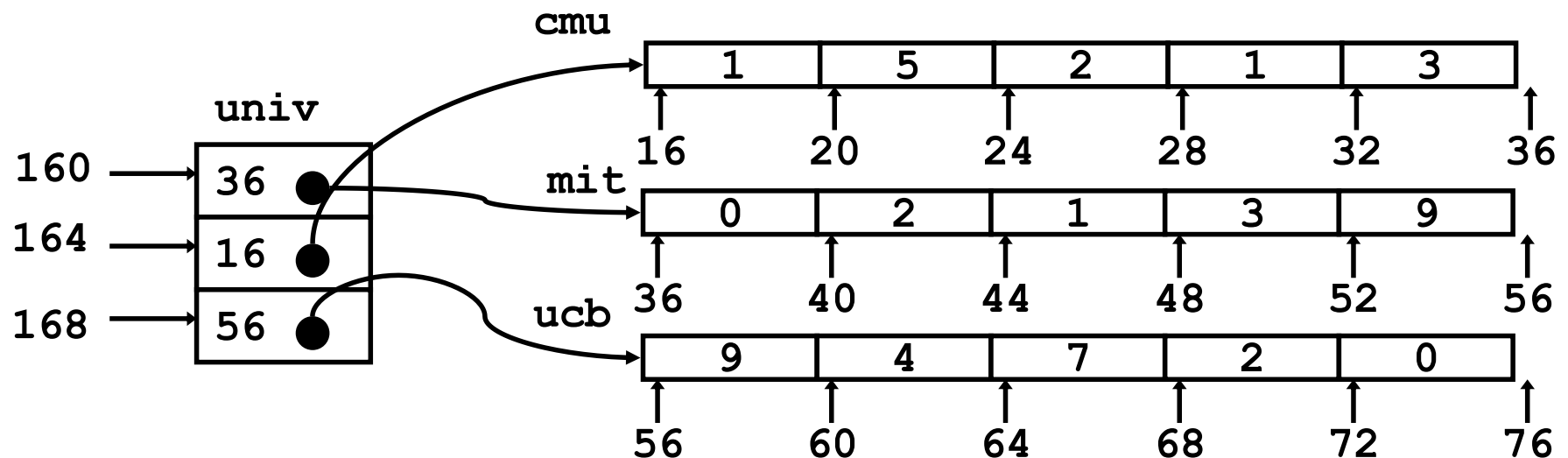
Array de apontadores para arrays: uma visão alternativa



- Variável `univ` é um *array* de 3 elementos
- Cada elemento:
 - um apontador de 4 *bytes*
 - aponta para um *array* de `int`'s

```
zip_dig cmu = { 1, 5, 2, 1, 3 };  
zip_dig mit = { 0, 2, 1, 3, 9 };  
zip_dig ucb = { 9, 4, 7, 2, 0 };
```

```
#define UCOUNT 3  
int *univ[UCOUNT] = {mit, cmu, ucb};
```



Array de apontadores para arrays: acesso a um elemento



Cálculo da localização

- para acesso a um elemento

$\text{Mem}[\text{Mem}[\text{univ} + 4 * \text{index}] + 4 * \text{dig}]$

- requer 2 acessos à memória

- um para buscar o apontador para *row array*
- outro para aceder ao elemento do *row array*

```
int get_univ_digit
(int index, int dig)
{
    return univ[index][dig];
}
```

```
                                # %ecx = index
                                # %eax = dig
leal 0(,%ecx,4),%edx           # 4*index
movl univ(%edx),%edx           # Mem[univ+4*index]
movl (%edx,%eax,4),%eax        # devolve Mem[Mem[univ+4*index]+4*dig]
```

Array de arrays versus array de apontadores para arrays



Modos distintos de cálculo da localização dos elementos:

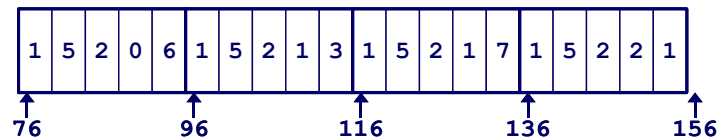
```
int get_pgh_digit
(int index, int dig)
{
    return pgh[index][dig];
}
```

```
int get_univ_digit
(int index, int dig)
{
    return univ[index][dig];
}
```

Array de arrays

- elemento em

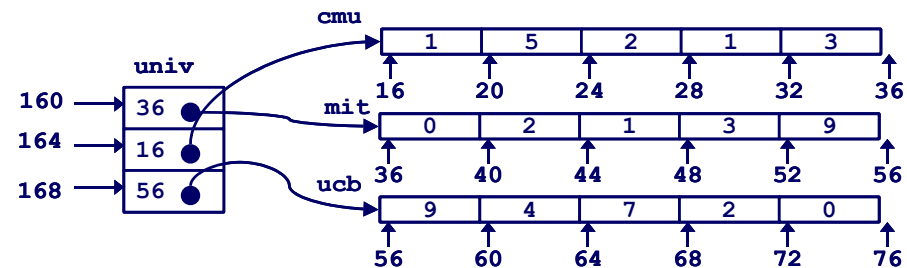
$\text{Mem}[\text{pgh} + 20 * \text{index} + 4 * \text{dig}]$



Array de apontadores para arrays

- elemento em

$\text{Mem}[\text{Mem}[\text{univ} + 4 * \text{index}] + 4 * \text{dig}]$



Arrays multi-dimensionais de tamanho fixo: a eficiência do compilador (1)

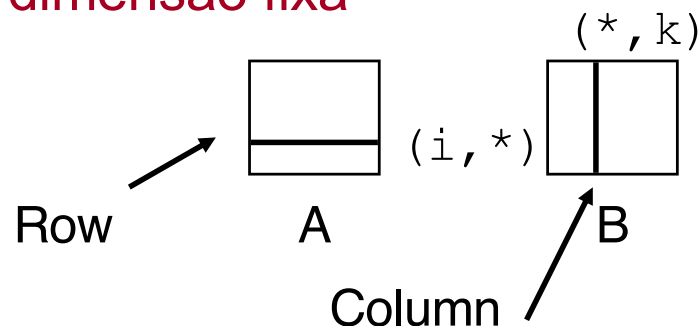


- Oportunidades para otimizar

- o *array* *a* está em localizações contíguas, começando em *a[i][0]*: usar apontador!
- o *array* *b* está em localizações espaçadas de $4*N$ células, começando em *b[0][j]*: usar também apontador!

- Limitações

- apenas funciona com arrays de dimensão fixa



```
#define N 16
typedef int fix_matrix[N][N];
```

```
/* Compute element i,k of
   fixed matrix product */
int fix_prod_ele
(fix_matrix a, fix_matrix b,
 int i, int k)
{
    int j;
    int result = 0;
    for (j = 0; j < N; j++)
        result += a[i][j]*b[j][k];
    return result;
}
```

Arrays multi-dimensionais de tamanho fixo: a eficiência do compilador (2)



- Otimizações automáticas do compilador:

–antes...

```
#define N 16
typedef int fix_matrix[N][N];
```

```
/* Compute element i,k of
   fixed matrix product */
int fix_prod_ele
(fix_matrix a, fix_matrix b,
 int i, int k)
{
    int j;
    int result = 0;
    for (j = 0; j < N; j++)
        result += a[i][j]*b[j][k];
    return result;
}
```

–depois...

```
/* Compute element i,k ... */
int fix_prod_ele (...)
{
    int *Aptr = &A[i][0];
    int *Bptr = &B[0][k];
    int cnt = N-1;
    int result = 0;
    do {
        result += (*Aptr)*(*Bptr);
        Aptr += 1;
        Bptr += N;
        cnt--;
    }while (cnt>=0);
    return result;
}
```

Structure: noções básicas

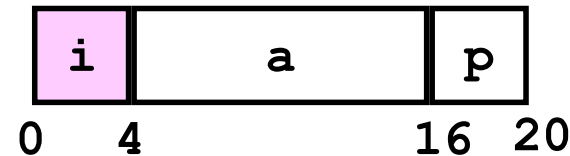


Propriedades

- em regiões contíguas da memória
- membros podem ser de tipos diferentes
- membros acedidos por nomes

```
struct rec {  
    int i;  
    int a[3];  
    int *p;  
};
```

Organização na memória



Acesso a um membro da *structure*

```
void set_i(struct rec *r,  
          int val)  
{  
    r->i = val;  
}
```

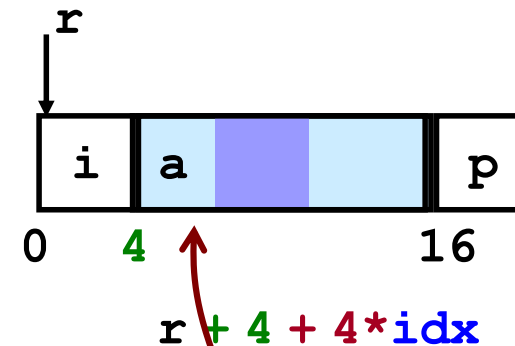
```
movl %eax, (%edx)    # %eax = val  
                    # %edx = r  
                    # Mem[r] = val
```

Structure: apontadores para membros (1)



```
struct rec {  
    int i;  
    int a[3];  
    int *p;  
};
```

```
int *find_a(struct rec *r, int idx)  
{  
    return &r->a[idx];  
}
```



Valor calculado
na compilação

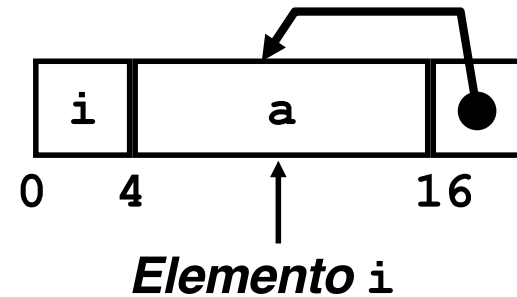
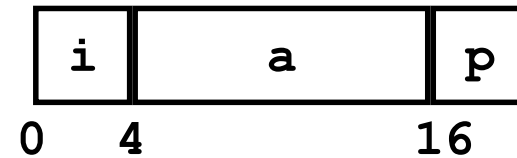
```
leal 4(%edx,%ecx,4),%eax    # %ecx= idx  
                             # %edx= r  
                             # r+4*idx+4
```

Structure: apontadores para membros (2)



```
struct rec {  
    int i;  
    int a[3];  
    int *p;  
};
```

```
void set_p(struct rec *r)  
{  
    r->p = &r->a[r->i];  
}
```



	# %edx = r
movl (%edx), %ecx	# r->i
leal 0(,%ecx,4), %eax	# 4*(r->i)
leal 4(%edx,%eax), %eax	# r+4+4*(r->i)
movl %eax, 16(%edx)	# Update r->p

Alinhamento de dados na memória



- **Dados alinhados**
 - Tipos de dados primitivos (escalares) requerem *K bytes*
 - Endereço deve ser múltiplo de *K*
 - Requisito nalgumas máquinas; aconselhado no IA-32
 - tratado de modo diferente, consoante Unix/Linux ou Windows!
- **Motivação para alinhar dados**
 - Memória acedida por *double* ou *quad-words* (alinhada)
 - ineficiente lidar com dados que passam esses limites
 - ainda mais crítico na gestão da memória virtual (limite da página!)
- **Compilador**
 - Insere bolhas na *structure* para garantir o correcto alinhamento dos campos

Alinhamento de dados na memória: os dados primitivos/escalares



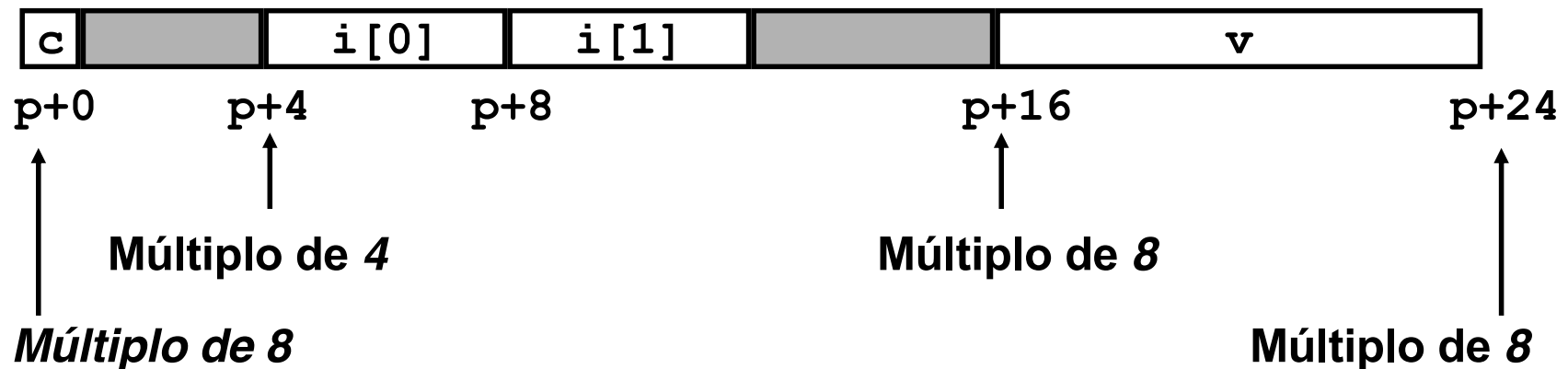
- 1 byte (e.g., `char`)
 - sem restrições no endereço
- 2 bytes (e.g., `short`)
 - o bit menos significativo do endereço deve ser 0_2
- 4 bytes (e.g., `int`, `float`, `char *`, etc.)
 - os 2 bits menos significativos do endereço devem ser 00_2
- 8 bytes (e.g., `double`)
 - Windows (e a maioria dos SO's & *instruction sets*):
 - os 3 bits menos significativos do endereço devem ser 000_2
 - Unix/Linux:
 - os 2 bits menos significativos do endereço devem ser 00_2
 - i.e., mesmo tratamento que um dado escalar de 4 bytes
- 12 bytes (`long double`)
 - Unix/Linux:
 - os 2 bits menos significativos do endereço devem ser 00_2
 - i.e., mesmo tratamento que um dado escalar de 4 bytes

Alinhamento de dados na memória: numa structure



- Deslocamentos dentro da *structure*
 - deve satisfazer os requisitos de alinhamento dos elementos (i.e., do seu maior elemento, K)
- Requisito para o endereço inicial
 - deve ser múltiplo de K
- Exemplo (em Windows):
 - $K = 8$, devido ao elemento `double`

```
struct S1 {  
    char c;  
    int i[2];  
    double v;  
} *p;
```



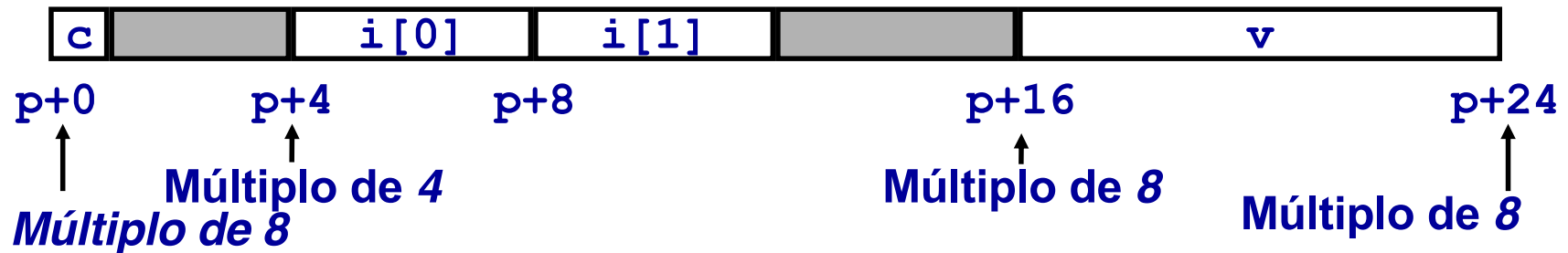
Alinhamento de dados na memória: Windows versus Unix/Linux



```
struct S1 {  
    char c;  
    int i[2];  
    double v;  
} *p;
```

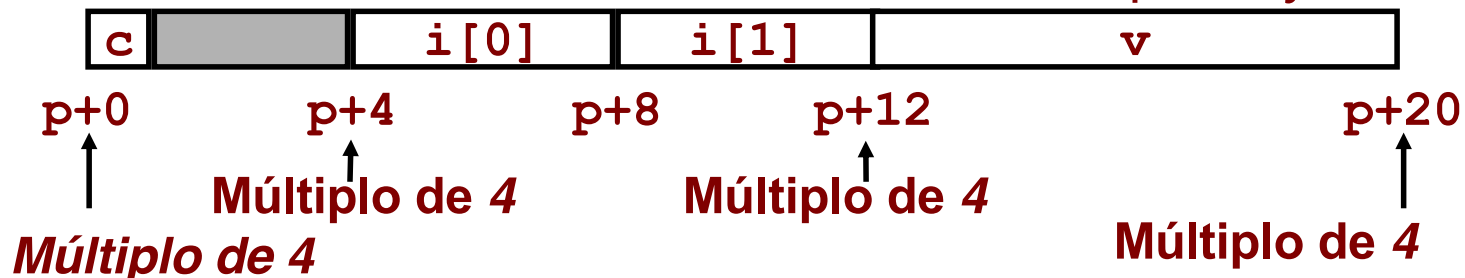
- **Windows:**

- $K = 8$, devido ao elemento `double`



- **Unix/Linux:**

- $K = 4$; `double` tratado como se fosse do tipo 4-bytes

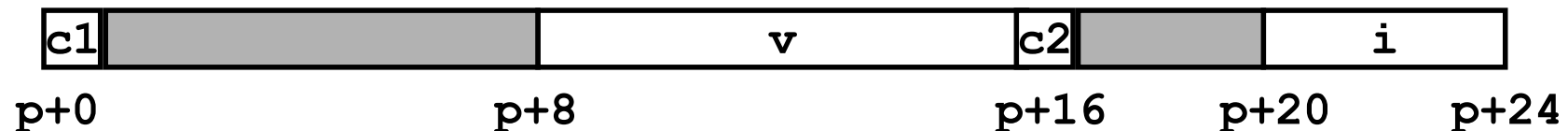


Alinhamento de dados na memória: ordenação dos membros



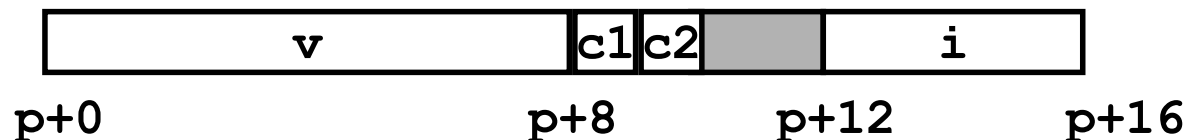
```
struct S4 {  
    char c1;  
    double v;  
    char c2;  
    int i;  
} *p;
```

**10 bytes de espaço desperdiçado
em Windows e em Unix/Linux**



```
struct S5 {  
    double v;  
    char c1;  
    char c2;  
    int i;  
} *p;
```

**apenas 2 bytes de espaço desperdiçado
em Windows e em Unix/Linux**



Union: noções básicas



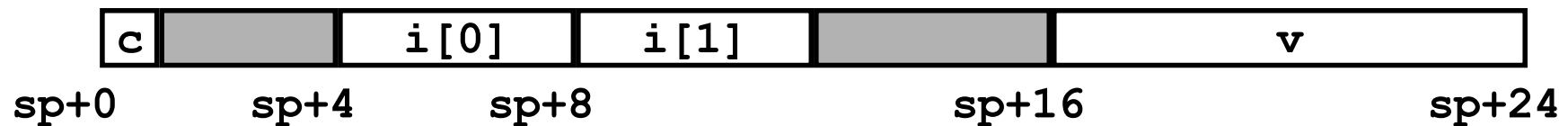
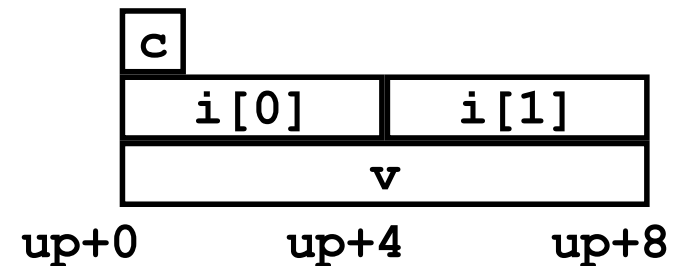
• Princípios

- sobreposição dos elementos de uma *union*
- memória alocada de acordo com o maior elemento
- só é possível aceder a um elemento de cada vez

```
struct S1 {  
    char c;  
    int i[2];  
    double v;  
} *sp;
```

```
union U1 {  
    char c;  
    int i[2];  
    double v;  
} *up;
```

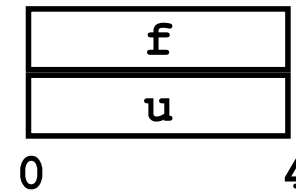
(alinhamento Windows)



Union: acesso a padrões de bits



```
typedef union {  
    float f;  
    unsigned u;  
} bit_float_t;
```



Como associar um padrão de bits,
a um dado `float`

```
float bit2float(unsigned u)  
{  
    bit_float_t arg;  
    arg.u = u;  
    return arg.f;  
}
```

isto NÃO é o mesmo que `(float) u`

Como obter o conjunto de bits
que representa um `float`

```
unsigned float2bit(float f)  
{  
    bit_float_t arg;  
    arg.f = f;  
    return arg.u;  
}
```

isto NÃO é o mesmo que `(unsigned) f`